Model Acceleration & Inference Optimization

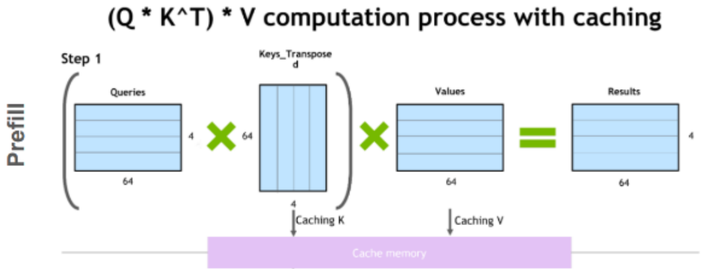
Fedor Velikonivtsev, 29 Oct 2026

Inference metrics, bottlenecks, GPU memory

One model, two inference phases

Prefill — process the prompt

- Process prompt tokens in parallel within each layer
- Compute and store their keys and values
- Produce the distribution for the first output token



https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization

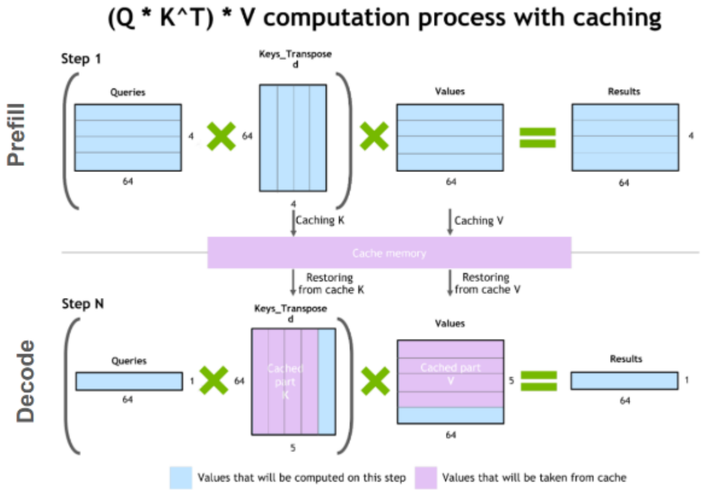
One model, two inference phases

Prefill — process the prompt

- Process prompt tokens in parallel within each layer
- Compute and store their keys and values
- Produce the distribution for the first output token

Decode — generate the continuation

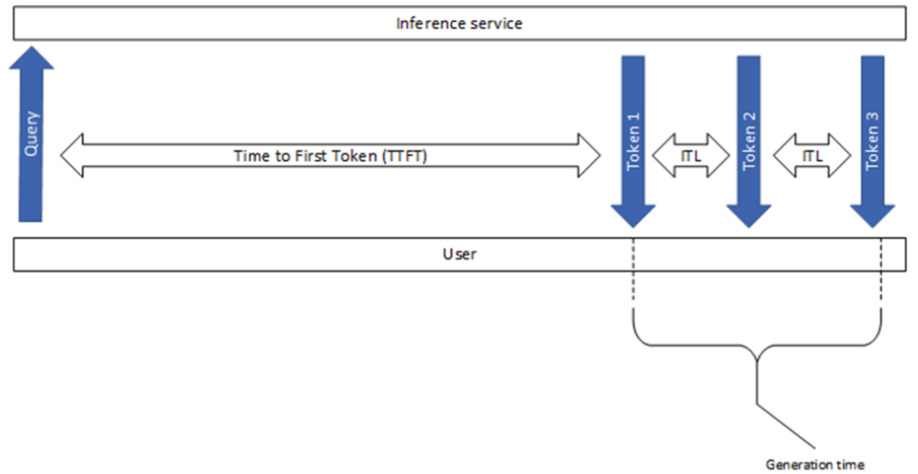
- Process one new token per sequence at each ordinary autoregressive step
- Attend to cached keys and values; append the new ones
- Repeat until generation stops



https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization

What does “Faster Inference” mean?

Time to first token (TTFT) — how long until the response starts?

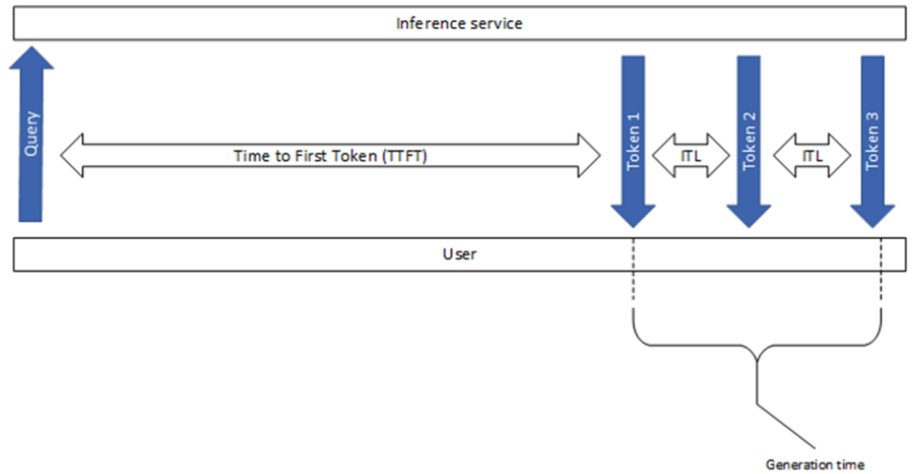


https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization

What does “Faster Inference” mean?

Time to first token (TTFT) — how long until the response starts?

Inter-token latency (ITL): how long between consecutive output tokens?



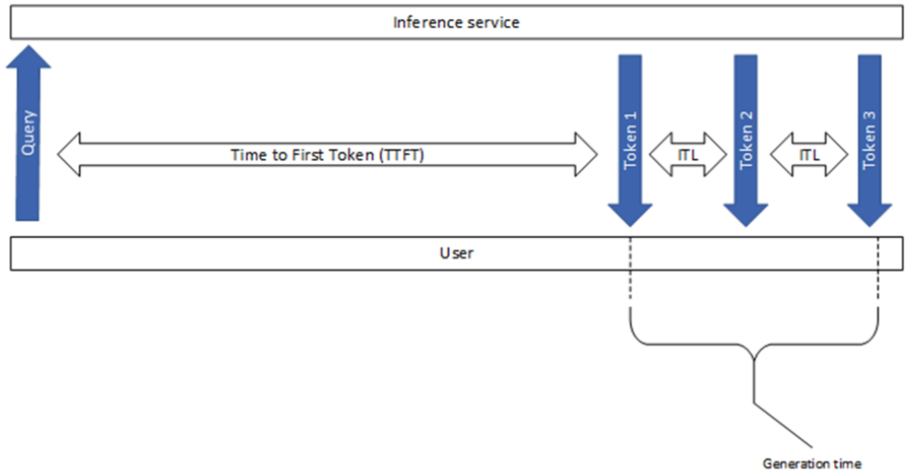
https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization

What does “Faster Inference” mean?

Time to first token (TTFT) — how long until the response starts?

Inter-token latency (ITL): how long between consecutive output tokens?

End-to-end latency: how long until the response finishes?



https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization

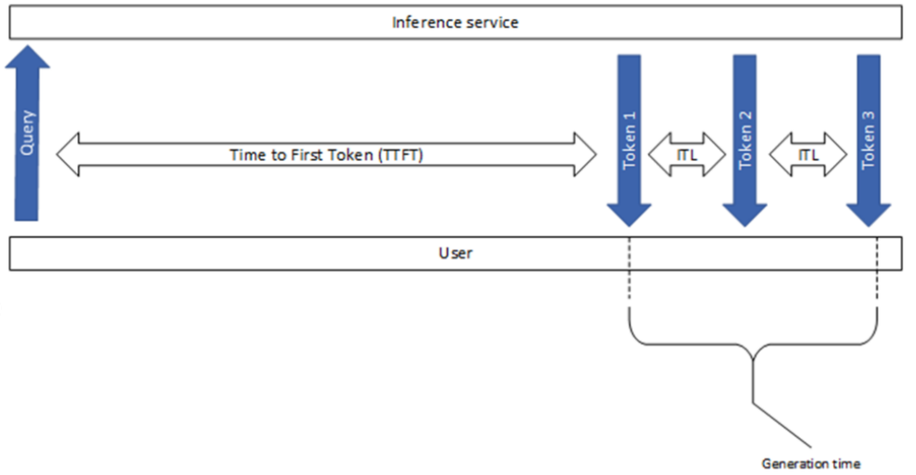
What does “Faster Inference” mean?

Time to first token (TTFT) — how long until the response starts?

Inter-token latency (ITL): how long between consecutive output tokens?

End-to-end latency: how long until the response finishes?

Output throughput: how many output tokens does the system deliver per second across all requests?



For one request with $N_{\text{out}} > 1$, average time per output token after the first is:

$$\text{TPOT} = \frac{T_{\text{end-to-end}} - \text{TTFT}}{N_{\text{out}} - 1}$$

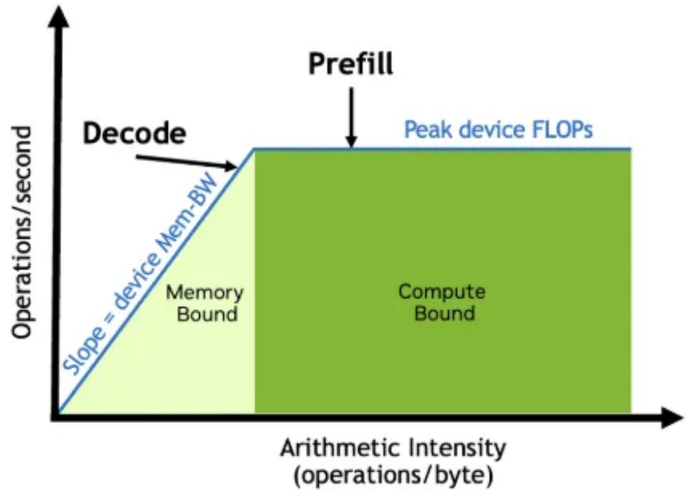
https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization

Compute-bound vs. memory-bound

A GPU must both **move data** and perform **arithmetic**

$$\text{Arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes transferred from GPU memory}},$$

$$\text{Attainable FLOP/s} \leq \min(\text{Peak FLOP/s}, \text{Memory bandwidth} \times \text{Arithmetic intensity})$$



https://developer.nvidia.com/blog/?p=120729

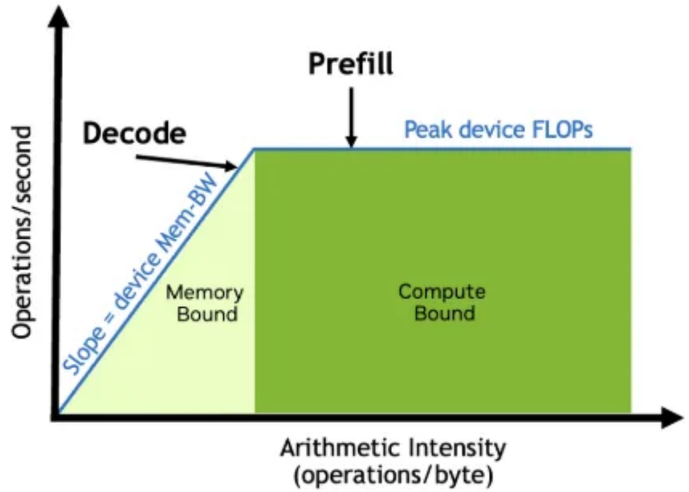
Compute-bound vs. memory-bound

A GPU must both **move data** and perform **arithmetic**

$$\text{Arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes transferred from GPU memory}},$$

$$\text{Attainable FLOP/s} \leq \min(\text{Peak FLOP/s}, \text{Memory bandwidth} \times \text{Arithmetic intensity})$$

Prefill has higher arithmetic intensity: many tokens reuse the same weights.



https://developer.nvidia.com/blog/?p=120729

Compute-bound vs. memory-bound

A GPU must both **move data** and perform **arithmetic**

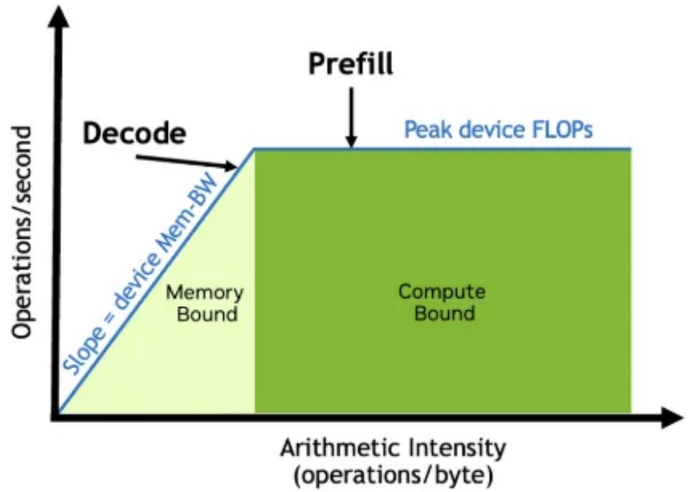
$$\text{Arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes transferred from GPU memory}},$$

$$\text{Attainable FLOP/s} \leq \min(\text{Peak FLOP/s}, \text{Memory bandwidth} \times \text{Arithmetic intensity})$$

Prefill has higher arithmetic intensity: many tokens reuse the same weights.

Small-batch decode is often bandwidth-bound: substantial data movement for relatively little computation.

Batching increases weight reuse; longer contexts increase KV-cache traffic.



https://developer.nvidia.com/blog/?p=120729

What occupies GPU memory

$M_{\text{total}} \approx$

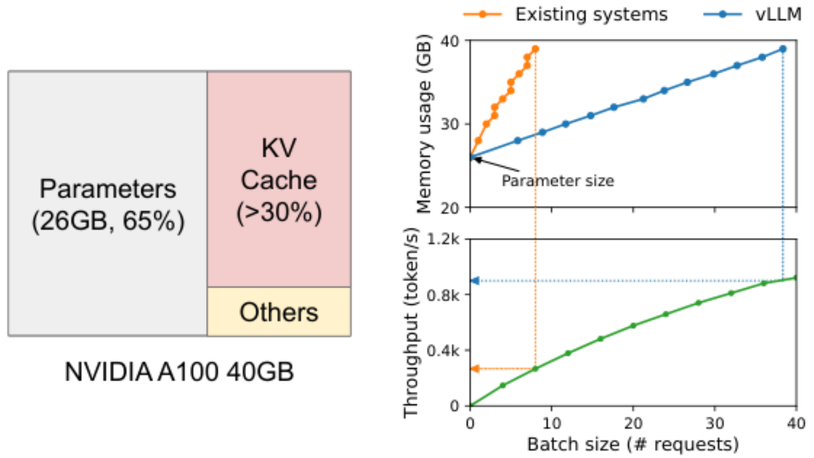


Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request.

https://arxiv.org/abs/2309.06180

What occupies GPU memory

$M_{\text{total}} \approx M_{\text{weights}} +$

Weights: persistent model parameters, shared across requests

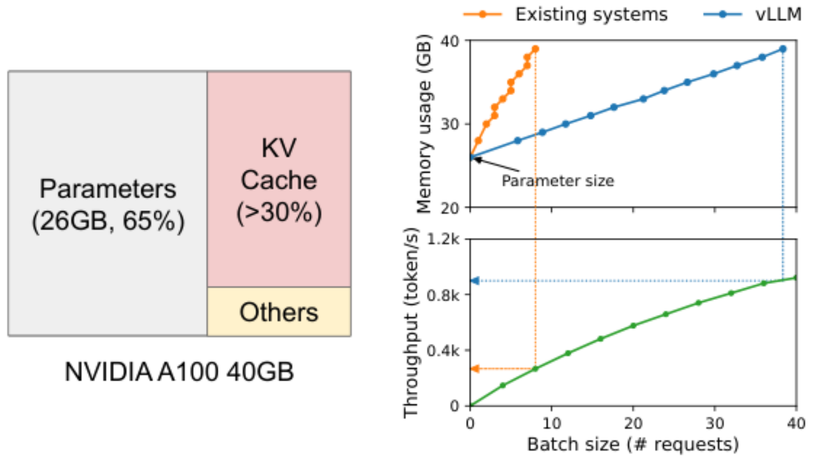


Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request.

https://arxiv.org/abs/2309.06180

What occupies GPU memory

$$M_{\text{total}} \approx M_{\text{weights}} + M_{\text{KV cache}} +$$

Weights: persistent model parameters, shared across requests

KV cache: stored attention state; grows with cached tokens and active sequences

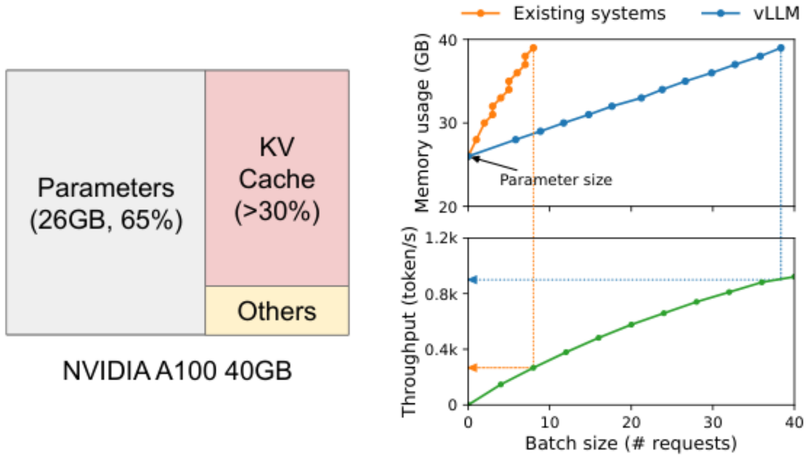


Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request.

https://arxiv.org/abs/2309.06180

What occupies GPU memory

$$M_{\text{total}} \approx M_{\text{weights}} + M_{\text{KV cache}} + M_{\text{activations/workspaces}} + M_{\text{runtime}}$$

Weights: persistent model parameters, shared across requests

KV cache: stored attention state; grows with cached tokens and active sequences

Activations and workspaces: temporary tensors and buffers used by kernels

Runtime overhead: framework state, allocation overhead, and other resources

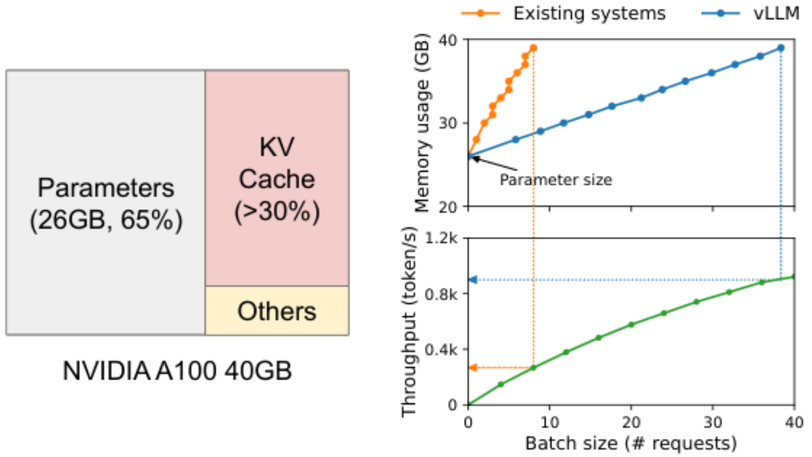


Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request.

https://arxiv.org/abs/2309.06180

What occupies GPU memory

$$M_{\text{total}} \approx M_{\text{weights}} + M_{\text{KV cache}} + M_{\text{activations/workspaces}} + M_{\text{runtime}}$$

Weights: persistent model parameters, shared across requests

KV cache: stored attention state; grows with cached tokens and active sequences

Activations and workspaces: temporary tensors and buffers used by kernels

Runtime overhead: framework state, allocation overhead, and other resources

Fitting the weights is only the first step. Serving also needs memory for requests

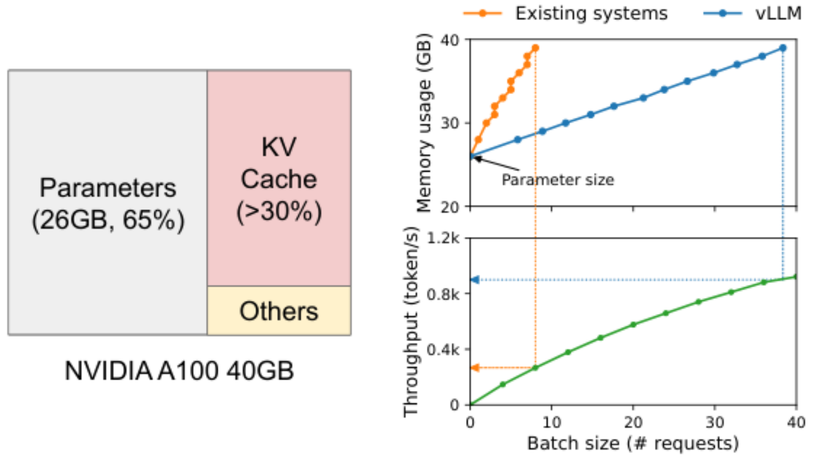


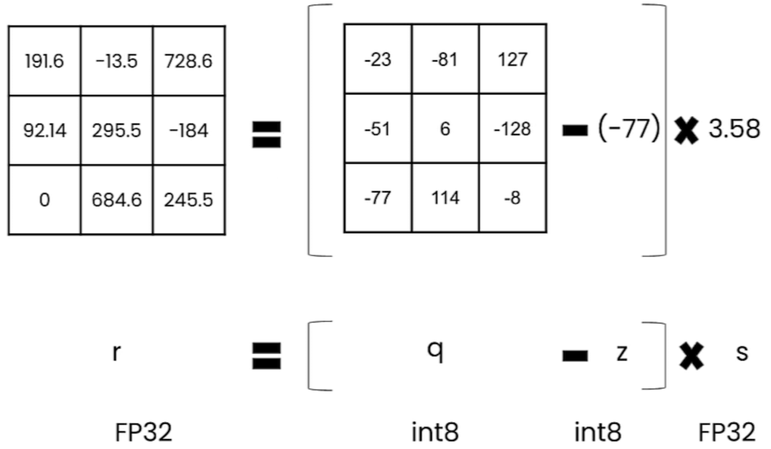
Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request.

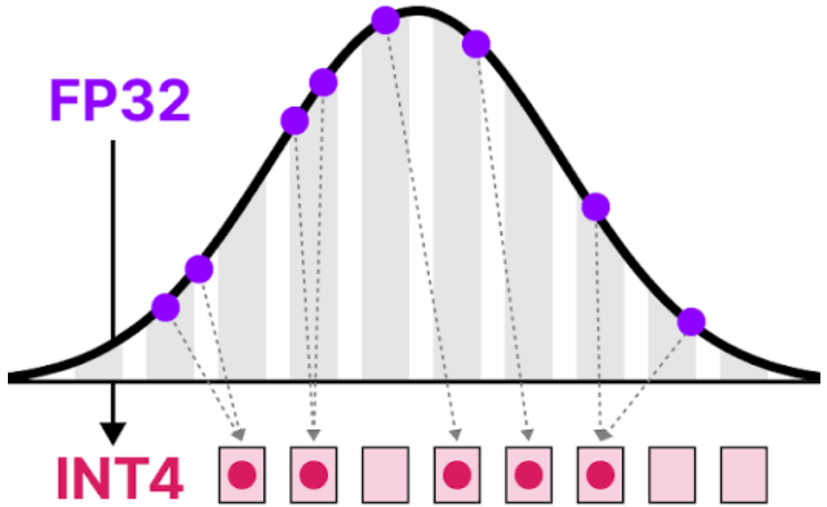
https://arxiv.org/abs/2309.06180

Quantization for inference

Fewer bits per value

Quantization approximates weights, activations, or cached states using a smaller set of representable values



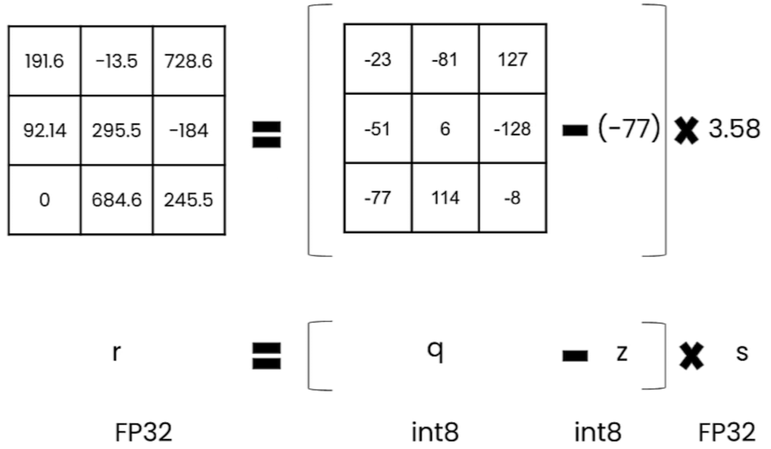


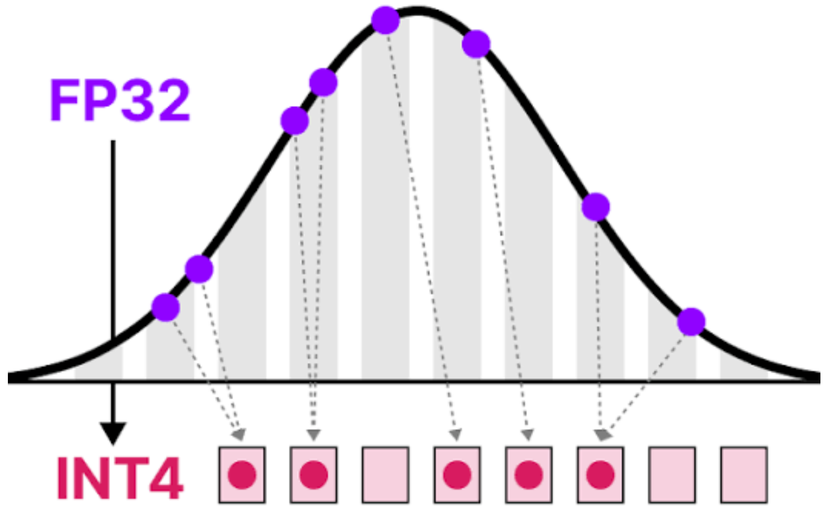
https://huggingface.co/docs/transformers/main/quantization/concept_guide

Fewer bits per value

Quantization approximates weights, activations, or cached states using a smaller set of representable values

The goal: reduce memory and latency while preserving model quality





https://huggingface.co/docs/transformers/main/quantization/concept_guide

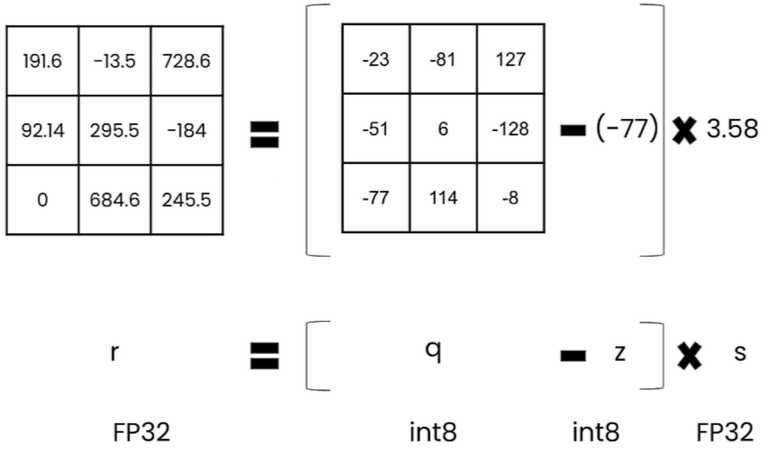
Fewer bits per value

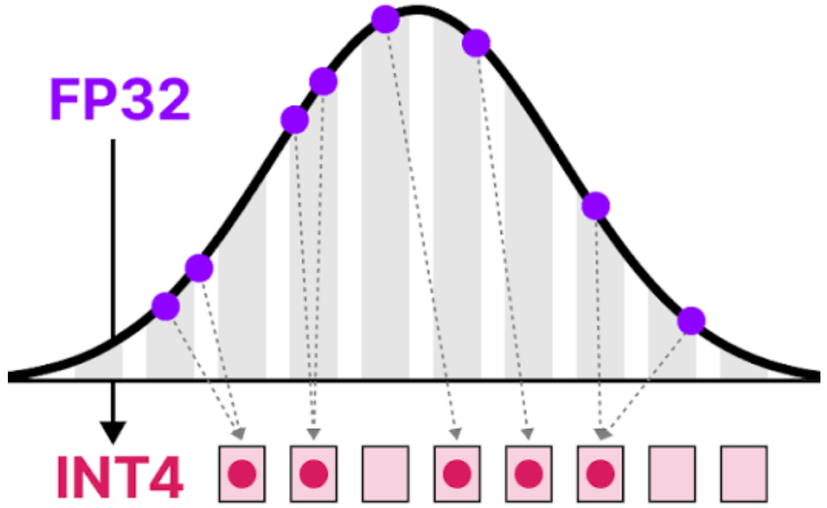
Quantization approximates weights, activations, or cached states using a smaller set of representable values

The goal: reduce memory and latency while preserving model quality

Why use fewer bits?

- **Memory capacity:** fit larger models or serve more requests
- **Memory bandwidth:** move fewer bytes for the same tensor
- **Compute:** use faster low-precision operations when supported by hardware and kernels





Fewer bits per value

Quantization approximates weights, activations, or cached states using a smaller set of representable values

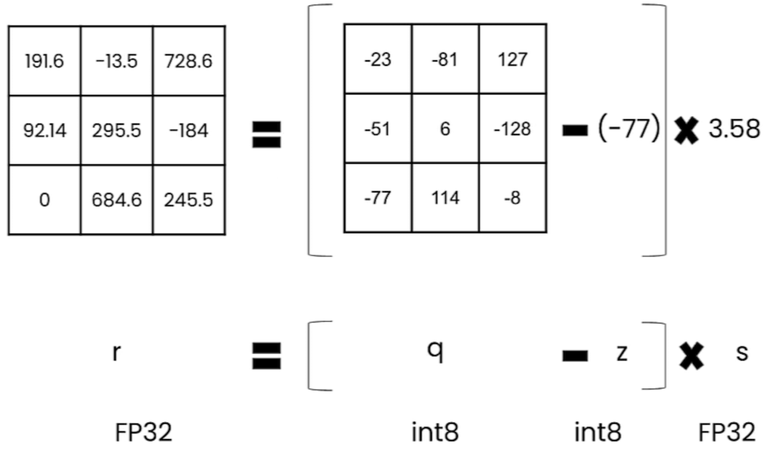
The goal: reduce memory and latency while preserving model quality

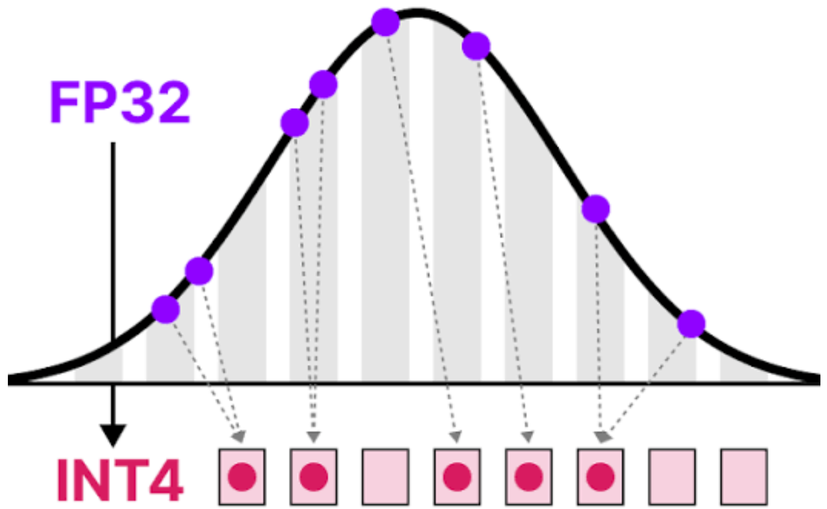
Why use fewer bits?

- **Memory capacity:** fit larger models or serve more requests
- **Memory bandwidth:** move fewer bytes for the same tensor
- **Compute:** use faster low-precision operations when supported by hardware and kernels

For P parameters stored at b bits each: $M_{\text{weights}} \approx \frac{P \cdot b}{8}$ bytes

- 70B parameters: BF16 → 140 GB, INT8 → 70 GB, INT4 → 35 GB





https://huggingface.co/docs/transformers/main/quantization/concept_guide

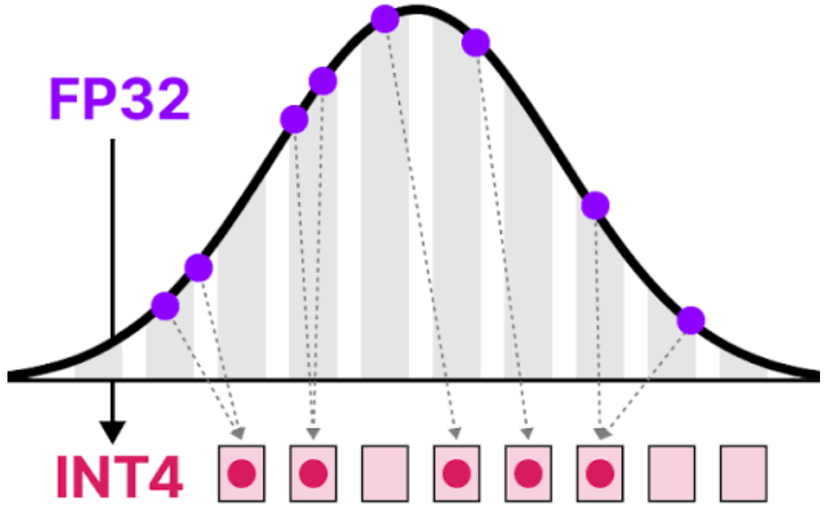
How quantization works

Store low-bit codes + shared parameters:

1. Choose the representation and scale

Example: INT4 with range [-7, 7]:

$$x = [-1.18,\ -0.36,\ 0.53,\ 1.40], \quad s = \frac{\max_i |x_i|}{7} = 0.20$$



https://huggingface.co/docs/transformers/main/quantization/concept_guide

How quantization works

Store low-bit codes + shared parameters:

1. Choose the representation and scale

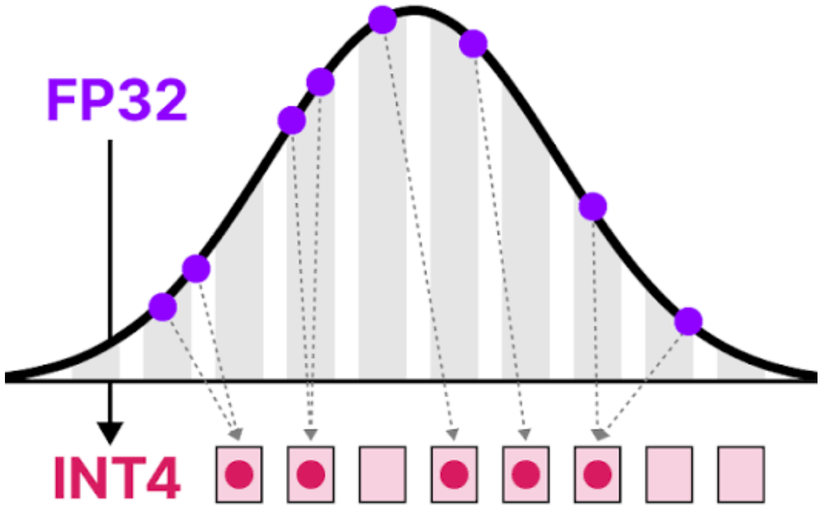
Example: INT4 with range [-7, 7]:

$$x = [-1.18,\ -0.36,\ 0.53,\ 1.40], \quad s = \frac{\max_i |x_i|}{7} = 0.20$$

2. Encode: scale, round and clip

$$q_i = \text{clip}\big(\text{round}(x_i/s), -7, 7\big)$$

$$[-1.18,\ -0.36,\ 0.53,\ 1.40] \xrightarrow{\div 0.20} [-5.9,\ -1.8,\ 2.65,\ 7] \xrightarrow{\text{round}} [-6,\ -2,\ 3,\ 7]$$



https://huggingface.co/docs/transformers/main/quantization/concept_guide

How quantization works

Store low-bit codes + shared parameters:

1. Choose the representation and scale

Example: INT4 with range [-7, 7]:

$$x = [-1.18,\ -0.36,\ 0.53,\ 1.40], \quad s = \frac{\max_i |x_i|}{7} = 0.20$$

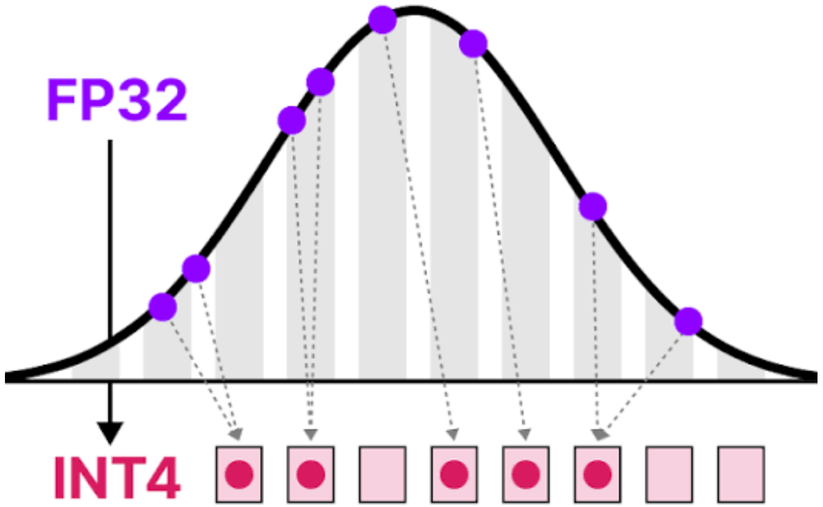
2. Encode: scale, round and clip

$$q_i = \mathrm{clip}\big(\mathrm{round}(x_i/s), -7, 7\big)$$

$$[-1.18,\ -0.36,\ 0.53,\ 1.40] \xrightarrow{\div 0.20} [-5.9,\ -1.8,\ 2.65,\ 7] \xrightarrow{\text{round}} [-6,\ -2,\ 3,\ 7]$$

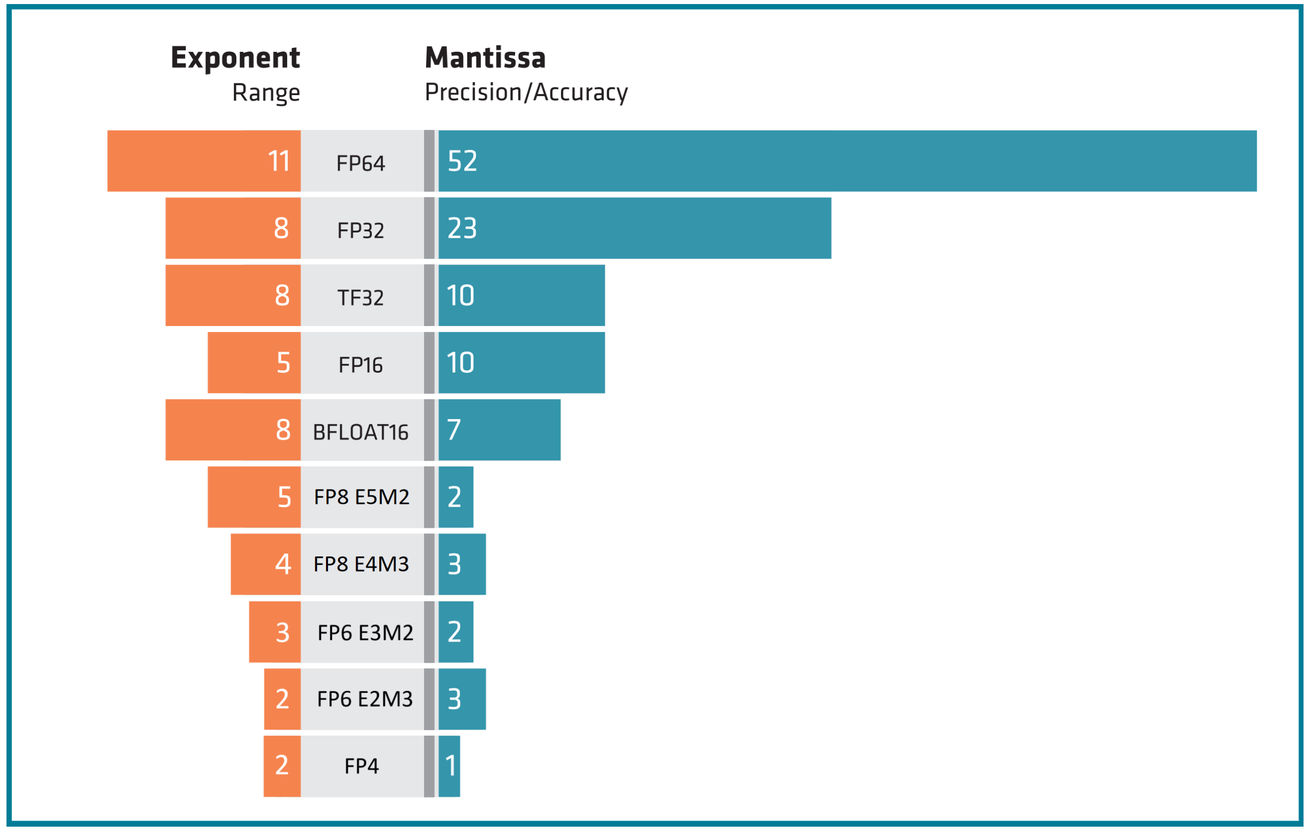
3. Store the codes, reconstruct approximate values when needed:

$$\hat{x} = s\,q = [-1.20,\ -0.40,\ 0.60,\ 1.40]$$



https://huggingface.co/docs/transformers/main/quantization/concept_guide

Floating-point types on GPU



https://rocm.docs.amd.com/en/latest/reference/precision-support.html

Quantization granularity

- **Per-tensor:** one scale for the entire tensor

Per-Tensor: 

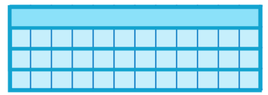
Quantization granularity

- **Per-tensor:** one scale for the entire tensor
- **Per-channel:** one scale per channel

Per-Tensor:

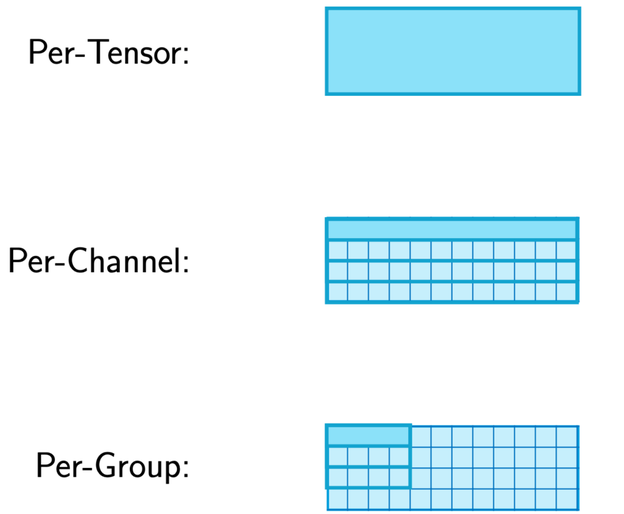


Per-Channel:



Quantization granularity

- **Per-tensor:** one scale for the entire tensor
- **Per-channel:** one scale per channel
- **Per-group / per-block:** one scale per small group of values



Quantization granularity

Microscaling: low-bit values with local shared scales

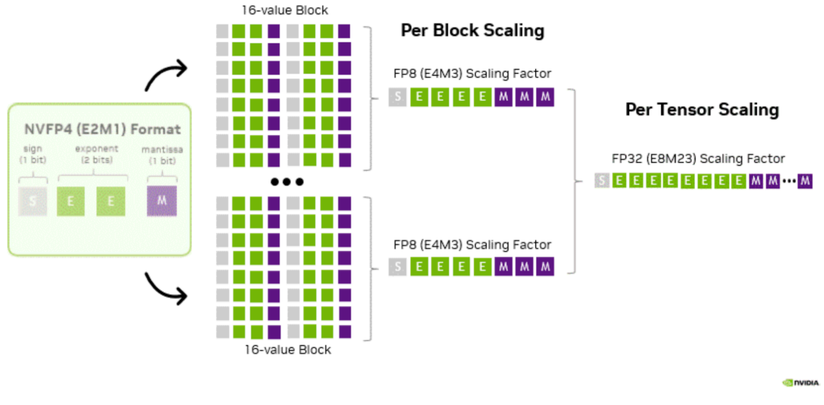
$$\hat{x}_i = s_{\text{block}(i)}\, q_i$$

- **MXFP8 / MXFP4:** 32 values share an 8-bit E8M0 power-of-two scale
- **NVFP4:** 16 FP4 values share an FP8 E4M3 scale, with an additional FP32 scale per tensor

Finer scaling adapts to local ranges, but requires more metadata

$$b_{\text{effective}} = b_{\text{value}} + \frac{b_{\text{scale}}}{G}$$

- **MXFP4:** 4+8/32=4.25 bits/value
- **NVFP4:** 4+8/16=4.5 bits/value
- Excludes padding and the additional per-tensor scale.

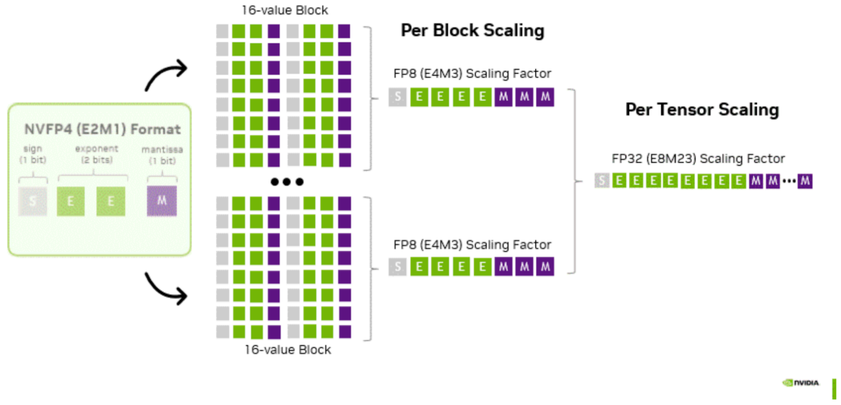


Quantization granularity

Microscaling: low-bit values with local shared scales

$$\hat{x}_i = s_{\text{block}(i)}\, q_i$$

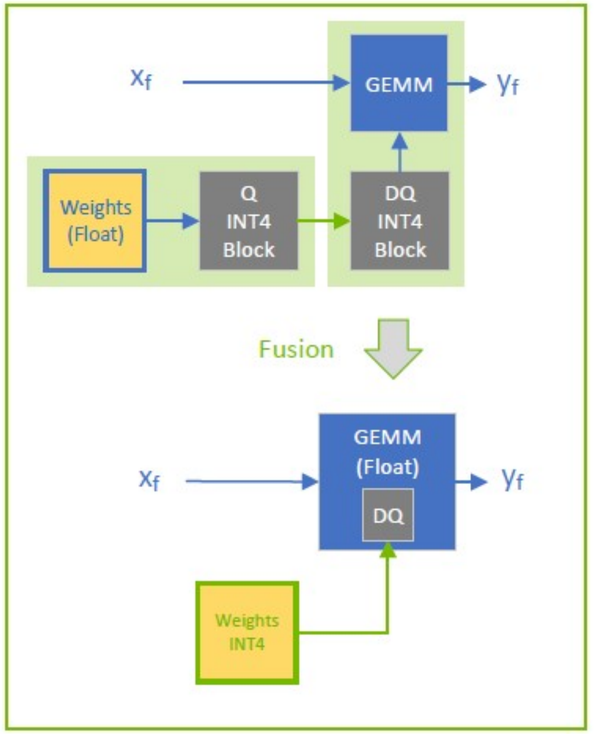
- **MXFP8 / MXFP4:** 32 values share an 8-bit E8M0 power-of-two scale
- **NVFP4:** 16 FP4 values share an FP8 E4M3 scale, with an additional FP32 scale per tensor



Weight-only vs. weight-and-activation quantization

Weight-only: example — W4A16

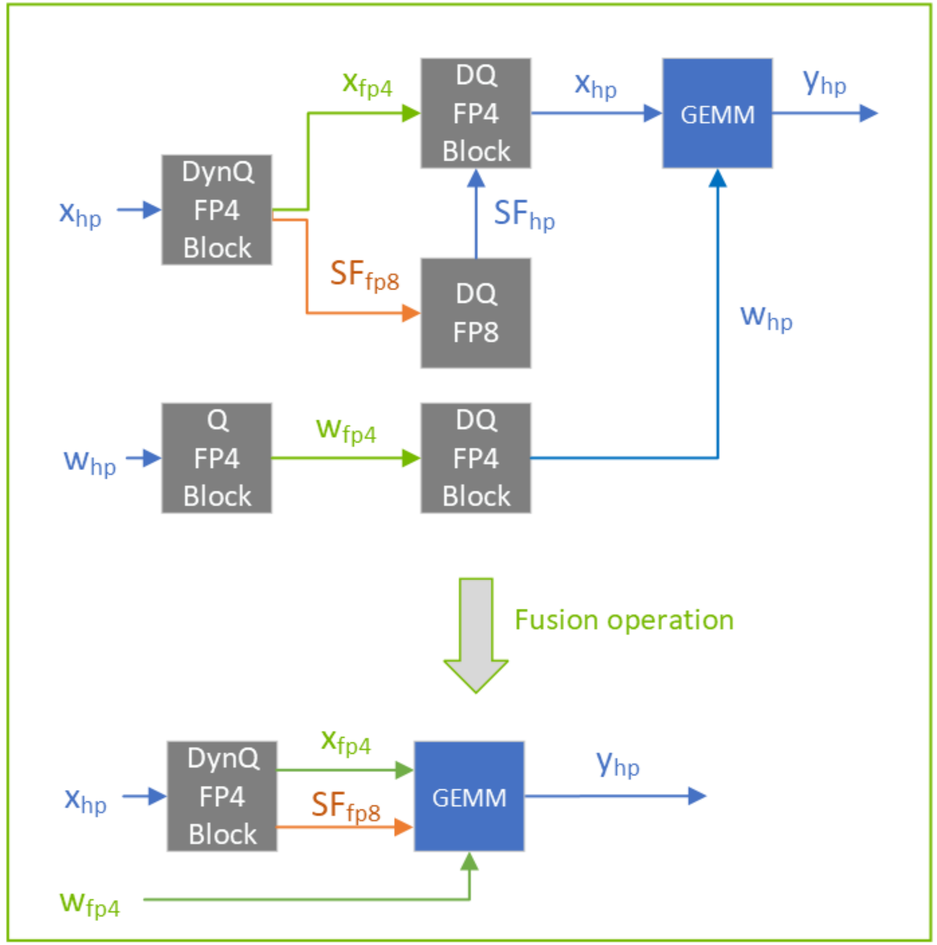
- Store weights in 4 bits; keep activations in FP16/BF16
- Unpack and dequantize weight tiles before multiplication
- Reduce weight storage and memory traffic



Weight-only vs. weight-and-activation quantization

Weight-and-activation: example — W8A8 or W4A4

- Quantize both operands of the matrix multiplication
- Use supported low-precision matrix instructions
- Activation quantization introduces additional error and runtime work



Outliers make quantization difficult

A few large values can determine the scale for many small values.

For symmetric INT4 with $s = \max|x|/7$:

$$[0.1, 0.2, 0.3, 10] \rightarrow [0, 0, 0, 7]$$

Outliers make quantization difficult

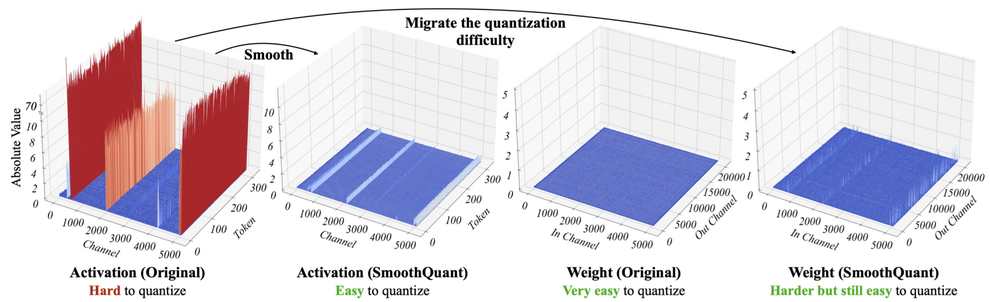
A few large values can determine the scale for many small values.

For symmetric INT4 with $s = \max|x|/7$:

$$[0.1, 0.2, 0.3, 10] \rightarrow [0, 0, 0, 7]$$

Some proposed ways to handle outliers:

- **LLM.int8():** compute outlier feature contributions in FP16; use INT8 for the remaining features.
- **SmoothQuant:** rescale channels to reduce activation outliers and absorb the inverse transformation into weights.
- For diagonal D: $XW = (XD^{-1})(DW)$



https://arxiv.org/abs/2208.07339

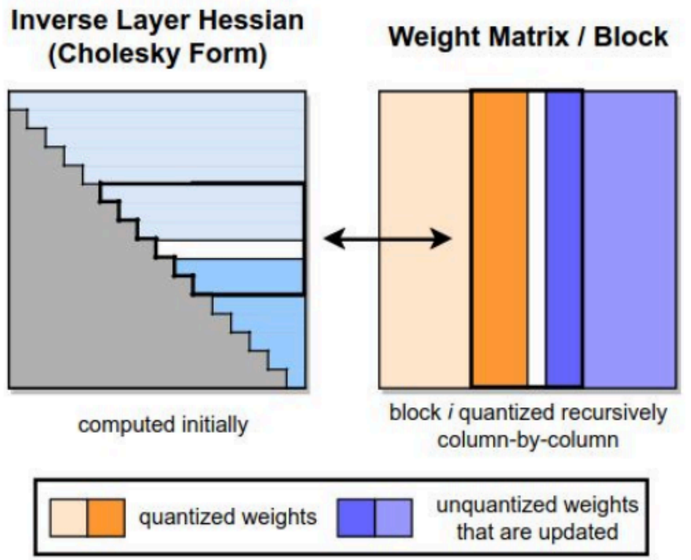
https://arxiv.org/abs/2211.10438

GPTQ

- Small weight error does not necessarily imply small output error
- For calibration activations X, GPTQ targets:

$$\min_{\hat{W}} \; \|XW - X\hat{W}\|_F^2$$

- Uses second-order information derived from calibration activations
- Compensates quantization errors by updating weights that have not yet been quantized



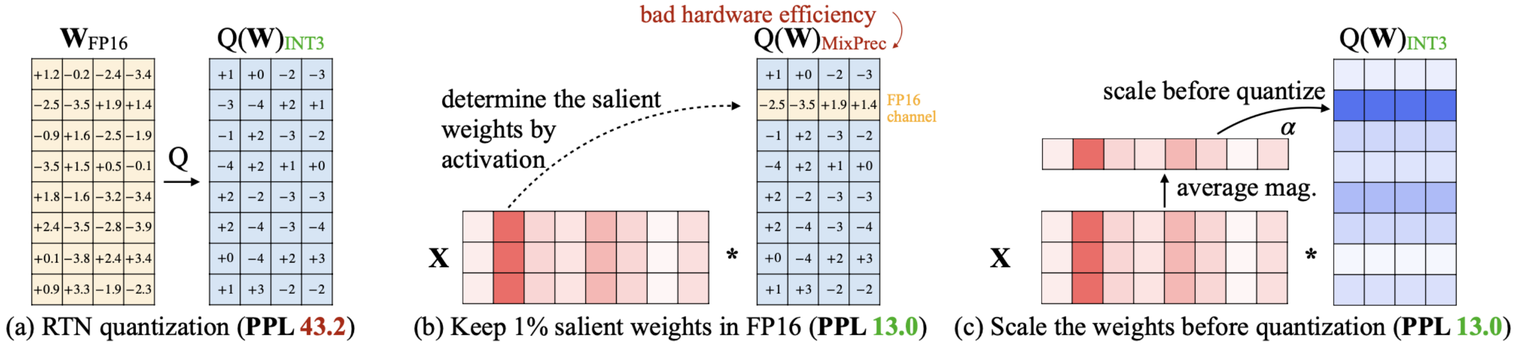
https://arxiv.org/abs/2210.17323

AWQ

- Uses activation statistics to identify important weight channels
- Searches channel scales that protect those weights during quantization
- Rescale sensitive weights during quantization

$$\underbrace{XW}_{\text{original}} = \underbrace{(XD^{-1})(DW)}_{\text{equivalent before quantization}} \approx \underbrace{(XD^{-1})\,\mathcal{Q}(DW)}_{\text{quantized weights}}$$

- D: positive diagonal matrix of input-channel scales
- $\mathcal{Q}$: quantization followed by dequantization



(a) RTN quantization (**PPL 43.2**) (b) Keep 1% salient weights in FP16 (**PPL 13.0**) (c) Scale the weights before quantization (**PPL 13.0**)

https://arxiv.org/abs/2306.00978

Rotations spread outliers across coordinates

- A vector can be difficult to quantize in one coordinate system and easier in another
- For an orthogonal matrix R:

$$RR^\top = I, \qquad XW = (XR)(R^\top W)$$

- **QuaRot:** uses randomized Hadamard transforms to reduce concentrated outliers
- **SpinQuant:** learns rotations that improve quantized-model quality
- Some transforms can be folded into weights; others require runtime computation
- The full-precision product is unchanged, but the quantization error changes.



https://arxiv.org/abs/2404.00456
https://arxiv.org/abs/2405.16406

Learning to tolerate quantization

Post-training quantization (PTQ) — quantize a pretrained model without additional model training

Quantization-aware training (QAT) — expose the model to quantization during training and approximate gradients through rounding

Quantization-aware distillation (QAD) — train the quantized student to match a high-precision teacher:

$$\mathscr{L}_{\text{QAD}} = D_{\text{KL}}\left(p_{\text{teacher}} \,\|\, p_{\text{student}}\right)$$

Examples

- **QuEST:** improved low-bit quantization and gradient estimation
- **Quartet II:** native NVFP4 forward and backward linear-layer computations
- **NVFP4 QAD:** recover inference quality through teacher supervision

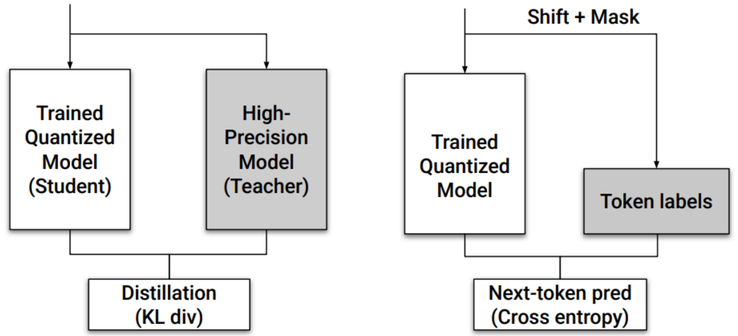


Figure 1 | Comparison of quantization-aware training (QAT) and quantization-aware distillation (QAD). QAT trains with next-token prediction (cross-entropy) on target datasets, while QAD uses distillation loss (KL divergence) with the full-precision teacher model providing soft targets.

https://arxiv.org/abs/2502.05003
https://arxiv.org/abs/2601.22813
https://arxiv.org/abs/2601.20088

KV-caches

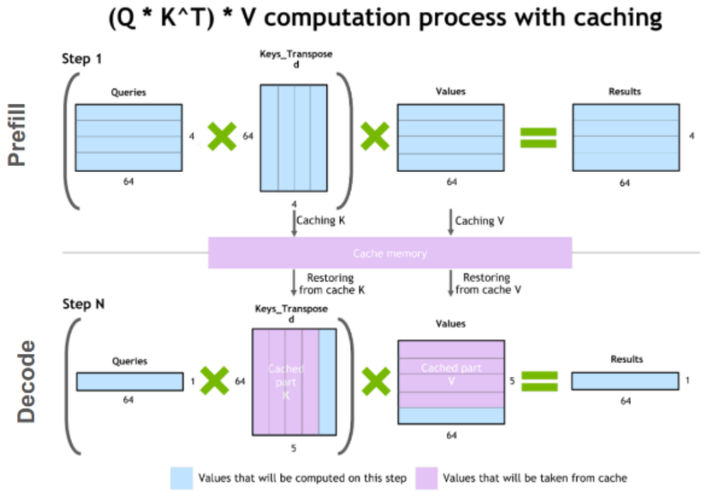
KV cache becomes bottleneck

The KV cache grows with both context length and the number of active requests.

For a standard Transformer with equally sized key and value heads:

$$M_{\mathrm{KV}} = 2\,L\,B\,T\,H_{\mathrm{KV}}\,d_h\,s$$

- L: number of layers
- B: active sequences, each with T cached tokens
- H_{KV}: KV heads per layer
- d_h: dimensions per head
- s: bytes per element



Example - 32 layers, 8 KV-heads, 128 dimension, BF16

- 1 seq × 8192 tokens — 1GiB
- 32 seq × 8192 tokens — 32 GiB
- 1 seq × 131072 tokens — 16 GiB

MHA vs MQA, GQA

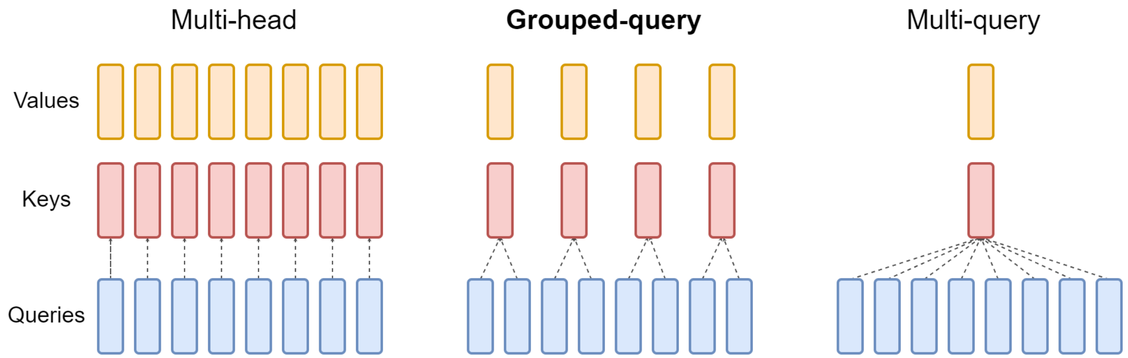
- Multiple query heads can share the same keys and values

MHA: each query head has its own K and V

GQA: each **group** of query heads shares K and V

MQA: all query heads share one K and one V

Fewer KV-heads reduce storage and can reduce memory traffic



Architecture	Query heads	KV heads	Relative
MHA	32	32	1x
GQA (group 4)	32	8	1/4
GQA (group 8)	32	4	1/8
MQA	32	1	1/32

https://arxiv.org/abs/2305.13245

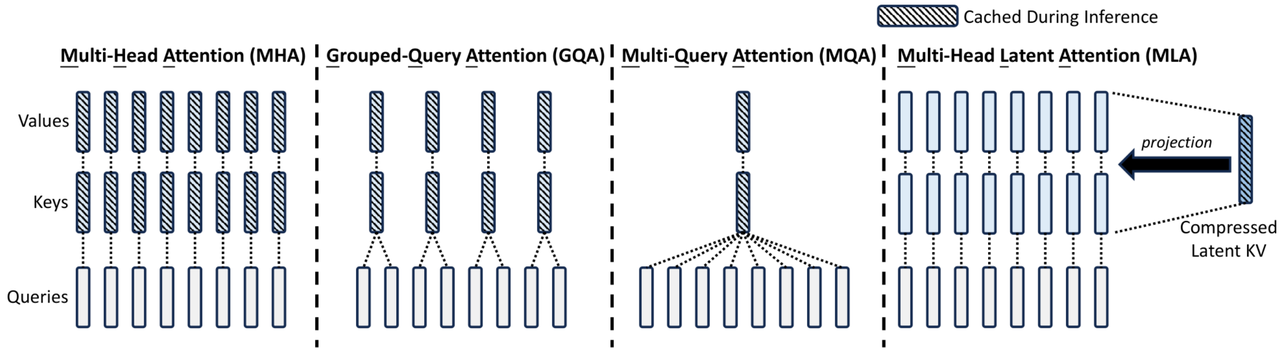
MLA — caching a latent representation

- Multihead Latent Attentiion represents keys and values through a shared latent vector

$$c_t = W_{\text{down}} h_t, \quad k_t^C = W_{\text{up}}^K c_t, \quad v_t = W_{\text{up}}^V c_t$$

- Cache per token, per layer:

$$\underbrace{c_t}_{\text{content latent}} + \underbrace{k_t^R}_{\text{positional key}}$$



Example:

$512 + 64 = 576$ elements

$576 \times 2 = 1{,}152$ bytes in BF16

https://arxiv.org/abs/2405.04434

MLA — caching a latent representation

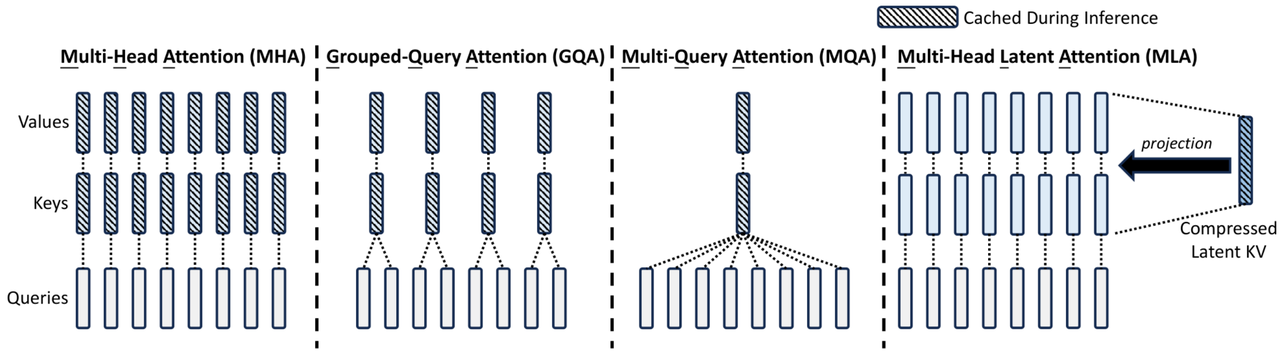
Linear projections can move outside the operations over cached tokens

Keys: transform the query

$$q^\top k_j = q^\top W_{\text{up}}^K c_j = \underbrace{\left((W_{\text{up}}^K)^\top q\right)^\top}_{\text{transformed query}} c_j$$

Values: aggregate before projecting

$$\sum_j a_j v_j = \sum_j a_j W_{\text{up}}^V c_j = W_{\text{up}}^V \left(\sum_j a_j c_j \right)$$



1. Read cached latent vectors
2. Compute attention scores and aggregate in latent space
3. Apply the value projection after aggregation

https://arxiv.org/abs/2405.04434

Native sparse attention

NSA combines three attention branches:

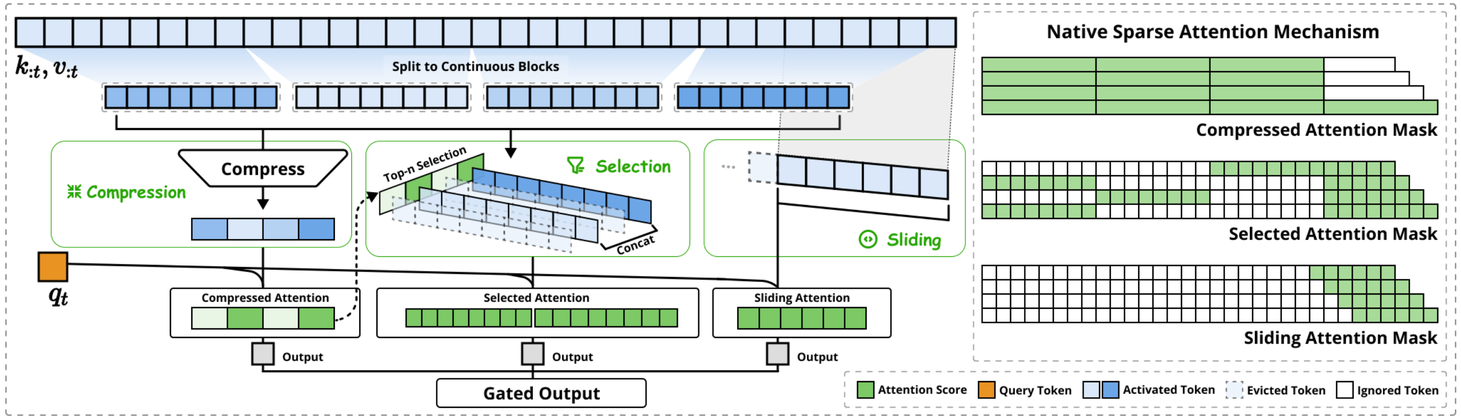
- **Compressed attention:** read coarse summaries of the history.
- **Selected attention:** read original tokens from relevant blocks.
- **Sliding-window attention:** preserve recent local detail

$$o_t = g_t^{\text{cmp}} o_t^{\text{cmp}} + g_t^{\text{sel}} o_t^{\text{sel}} + g_t^{\text{win}} o_t^{\text{win}}$$

Learned gates combine the branch outputs

Block selection and specialized kernels make sparsity useful on GPUs

The model learns with sparse attention during training



https://arxiv.org/abs/2502.11089

DeepSeek sparse attention

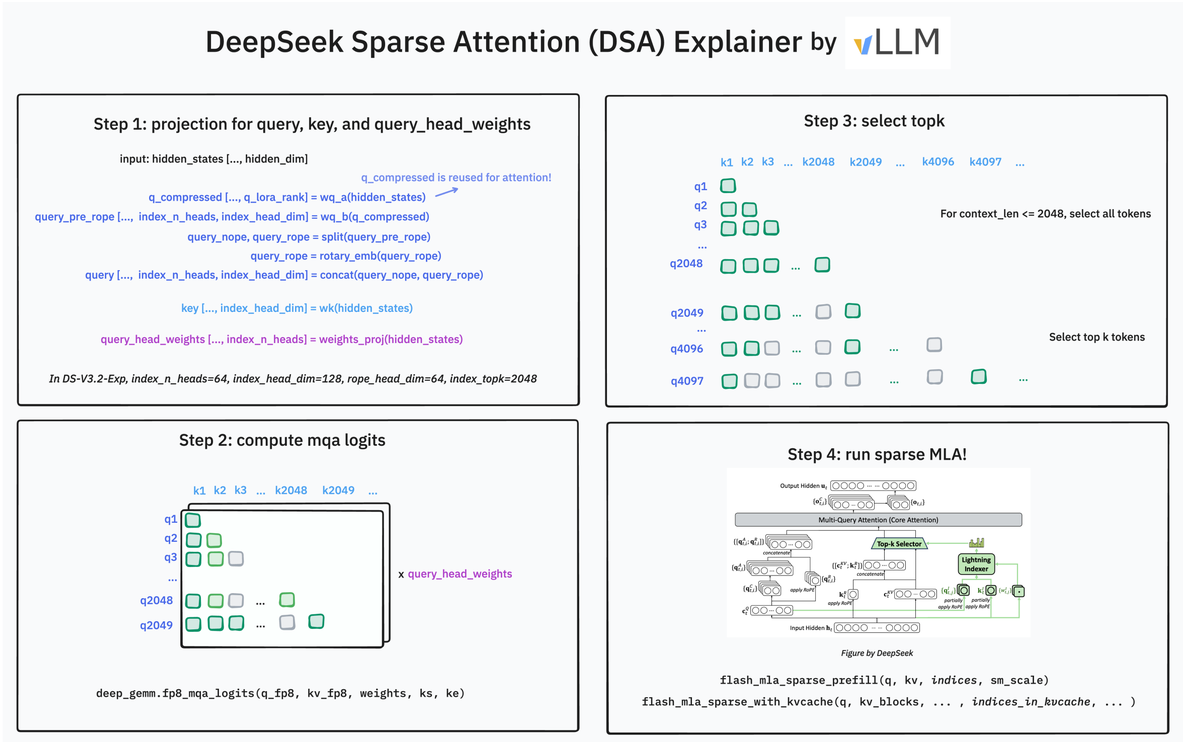
DSA uses a lightweight indexer to select entries for the main attention operation

- Score previous positions using compact indexer keys
- Select the top-K positions
- Run main attention over the selected MLA cache entries

$$\mathcal{I}_t = \mathrm{TopK}\big(\mathrm{Indexer}(q_t)\big), \quad o_t = \mathrm{Attention}\left(q_t, K_{\mathcal{I}_t}, V_{\mathcal{I}_t}\right)$$

Example: $k = 2{,}048$ at a 131,072-token context

Main attention selects approximately **1.6% of positions**.



https://vllm-project.github.io/2025/09/29/deepseek-v3-2.html

DeepSeek-V4 with compressed history

- Compressed Sparse Attention
  - Compress KV of m tokens into one representation
  - Apply DeepSeek Sparse Attention
- Heavily Compressed Attention
  - Even more compression

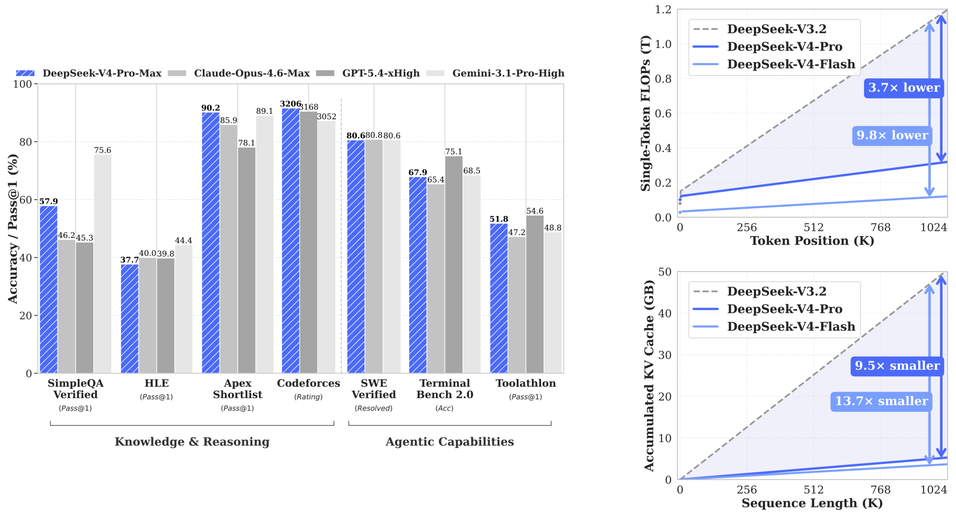


Figure 1 | **Left**: benchmark performance of DeepSeek-V4-Pro-Max and its counterparts. **Right**: inference FLOPs and KV cache size of DeepSeek-V4 series and DeepSeek-V3.2.

https://arxiv.org/abs/2606.19348

DeepSeek-V4 with compressed history

- Compressed Sparse Attention
  - Compress KV of m tokens into one representation
  - Apply DeepSeek Sparse Attention
- Heavily Compressed Attention
  - Even more compression

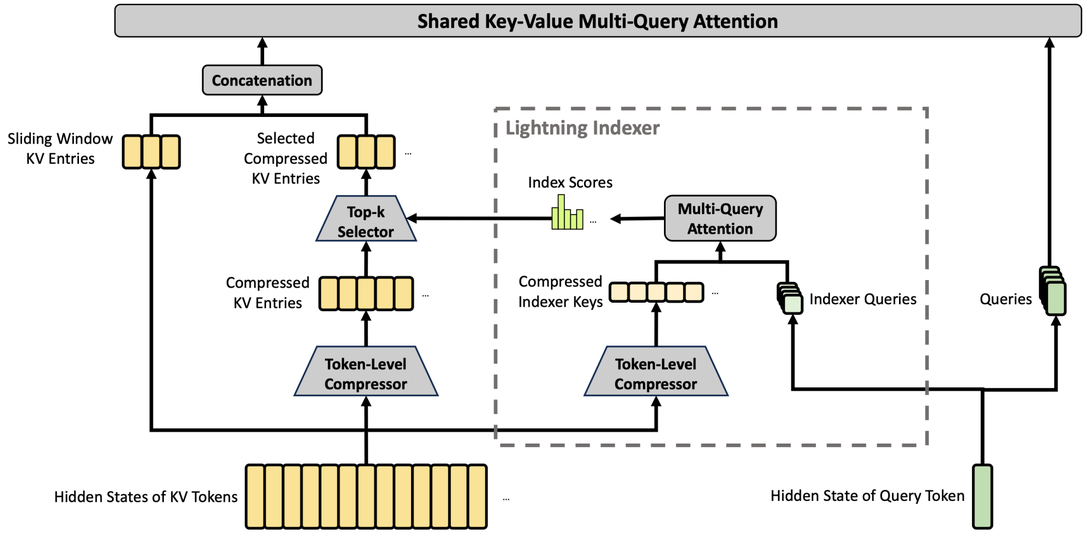


Figure 3 | Core architectures of CSA. It compresses the number of KV entries to $\frac{1}{m}$ times, and then applies DeepSeek Sparse Attention for further acceleration. Additionally, a small set of sliding window KV entries is combined with the selected compressed KV entries to enhance local fine-grained dependencies.

https://arxiv.org/abs/2606.19348

DeepSeek-V4.1

Compressed Sparse Attention 2

- **Full:** compute main KV, indexer keys, and Top-K indices
- **Reindex:** reuse KV and indexer keys, recompute Top-K indices
- **Reuse:** reuse both KV and Top-K indices.
- Each layer keeps its own query and sliding-window KV

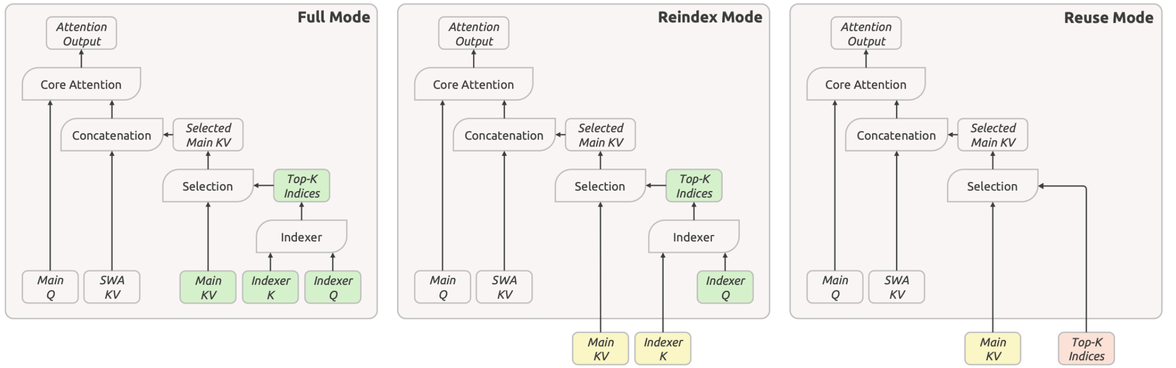


Figure 4 | **Three operating modes of CSA2.** The modes differ in how they obtain main KV, indexer K, and Top-K indices. Green blocks indicate quantities computed in the current layer; yellow blocks indicate main KV and indexer K reused from the most recent Full Mode layer; while red blocks indicate Top-K indices reused from the most recent index-producing (Full or Reindex Mode) layer. All three modes compute main Q and SWA KV in the current layer.

https://arxiv.org/abs/2606.19348

DeepSeek-V4.1

Hierarchical Sparse Indexer

- The first Full-mode decoder layer scans the available history and builds a shared candidate pool
- Later Reindex layers select their own **Top-512** entries from this pool of up to **16,384 positions**
- This bounds later indexing cost independently of context length

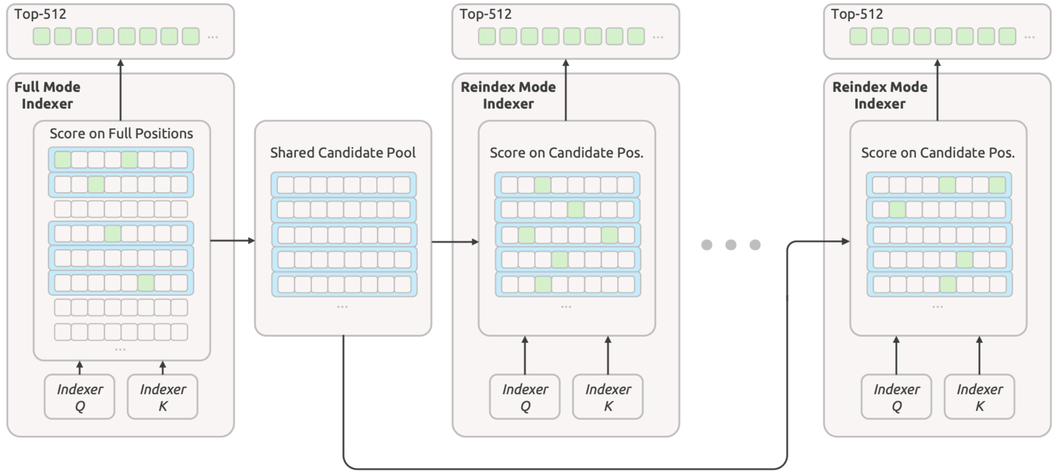


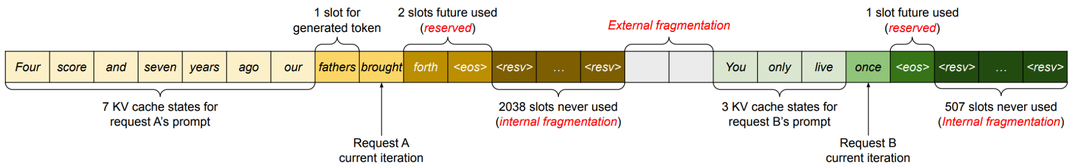
Figure 5 | **Hierarchical Sparse Indexer.** Each square represents a position; green squares mark selected indices, and blue rectangles mark blocks selected based on their maximum indexer scores. The decoder's first CSA2 layer in Full mode selects its own Top-512 indices and builds a shared candidate pool from the selected blocks for subsequent layers. CSA2 layers in Reindex mode then select their Top-512 indices from this pool.

https://arxiv.org/abs/2606.19348

Paged Attention

PagedAttention stores a sequence's KV cache in fixed-size blocks

- Logical token order stays contiguous
- Physical blocks can occupy different locations in GPU memory
- A block table maps logical blocks to physical blocks
- New blocks are allocated as the sequence grows



Paging reduces allocation waste while preserving the attention computation

Example: 10 tokens, 4 tokens per block

$$\left\lceil \frac{10}{4} \right\rceil = 3 \text{ blocks}$$

12 allocated token slots, 10 used, 2 unused

https://arxiv.org/abs/2309.06180

Paged Attention

PagedAttention stores a sequence's KV cache in fixed-size blocks

- Logical token order stays contiguous
- Physical blocks can occupy different locations in GPU memory
- A block table maps logical blocks to physical blocks
- New blocks are allocated as the sequence grows

Paging reduces allocation waste while preserving the attention computation

Example: 10 tokens, 4 tokens per block

$$\left\lceil \frac{10}{4} \right\rceil = 3 \text{ blocks}$$

12 allocated token slots, 10 used, 2 unused

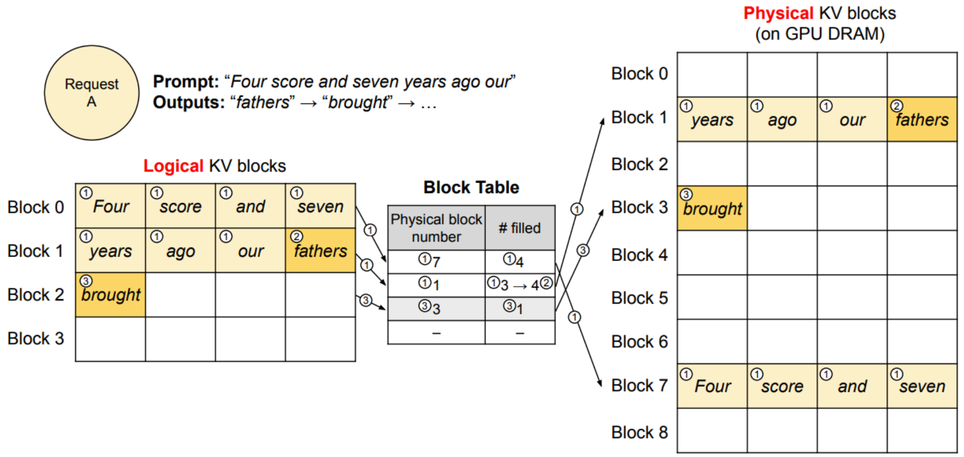


Figure 6. Block table translation in vLLM.

https://arxiv.org/abs/2309.06180

Paged Attention

PagedAttention stores a sequence's KV cache in fixed-size blocks

- Logical token order stays contiguous
- Physical blocks can occupy different locations in GPU memory
- A block table maps logical blocks to physical blocks
- New blocks are allocated as the sequence grows

Paging reduces allocation waste while preserving the attention computation

Example: 10 tokens, 4 tokens per block

$$\left\lceil \frac{10}{4} \right\rceil = 3 \text{ blocks}$$

12 allocated token slots, 10 used, 2 unused

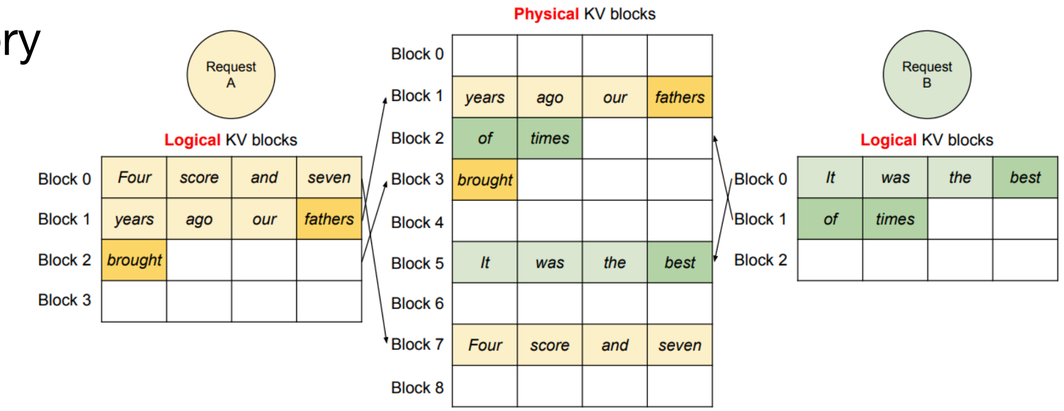


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

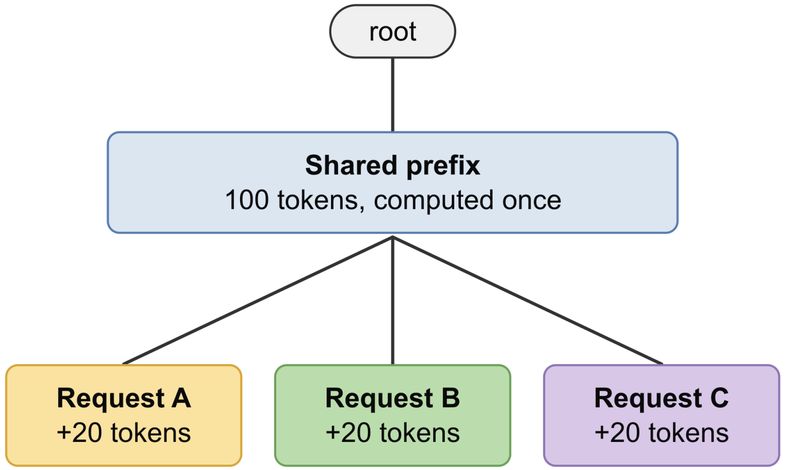
https://arxiv.org/abs/2309.06180

Radix Attentiion

RadixAttention stores KV cache in a radix tree, indexed by token prefix

- Tree edges are labeled with token sequences
- Each node stores the KV cache of its tokens
- A new request reuses its longest cached prefix
- Least recently used leaves are evicted first

Prefix reuse skips recomputation while preserving the attention computation



Each request computes only its own suffix

https://arxiv.org/abs/2312.07104

Radix Attention

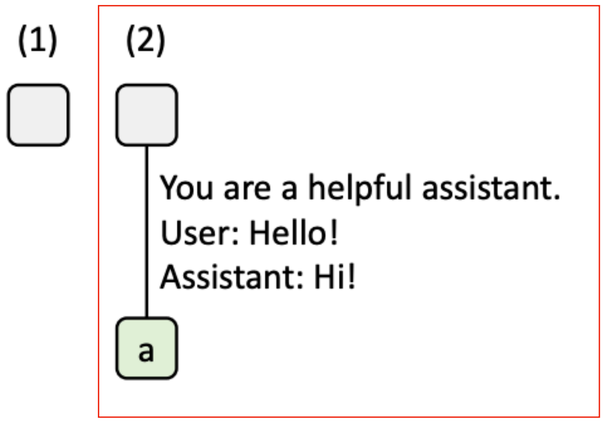
In step (1), the radix tree is initially empty



Radix Attention

In step (2), the server processes an incoming user message "Hello" and responds with the LLM output “Hi"

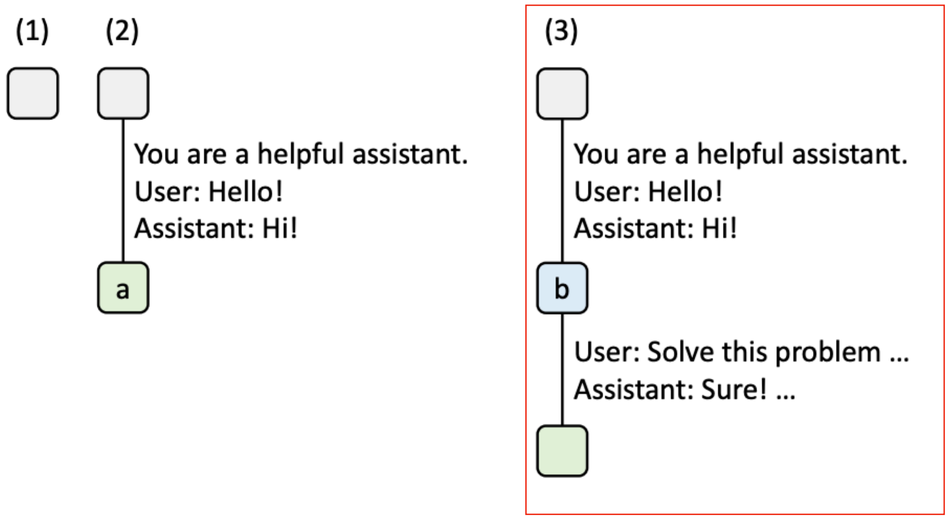
The system prompt "You are a helpful assistant", the user message "Hello!", and the LLM reply "Hi!" are consolidated into the tree as a single edge linked to a new node



Radix Attention

In step (3), a new prompt arrives and the server finds the prefix of the prompt (i.e., the first turn of the conversation) in the radix tree and reuses its KV cache

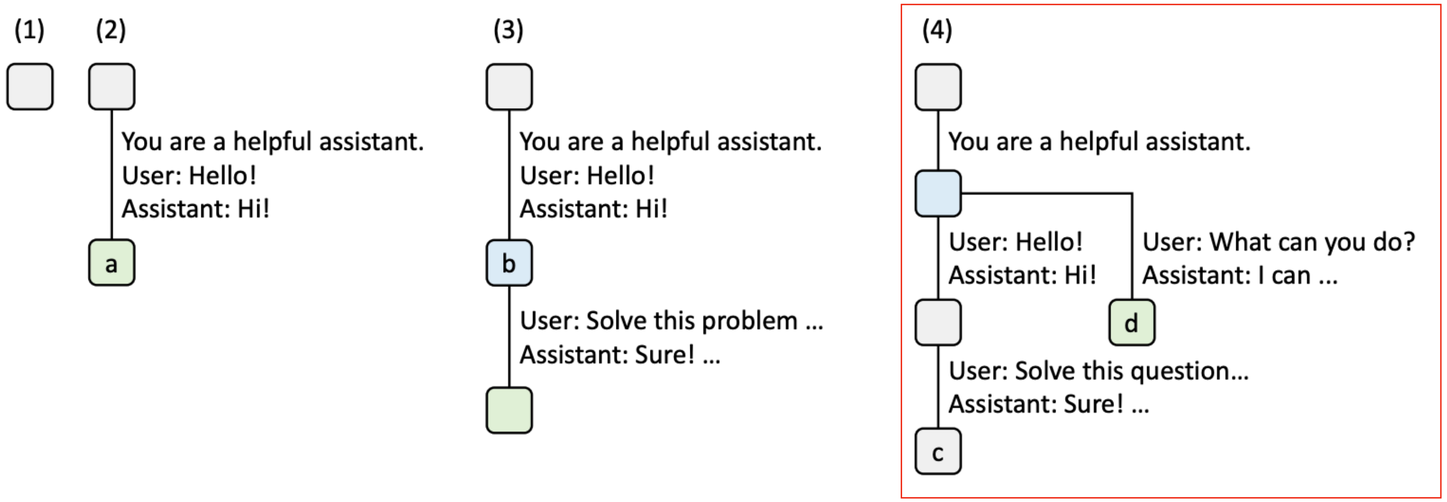
The new turn is appended to the tree as a new node



Radix Attention

In step (4), a new chat session begins

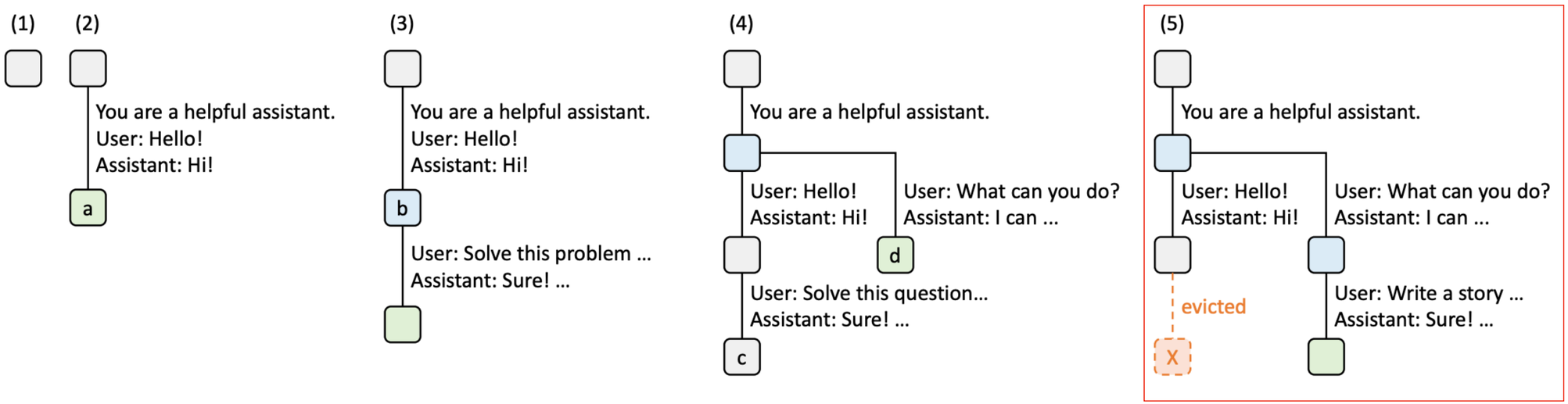
The node “b” from (3) is split into two nodes to allow the two chat sessions to share the system prompt



Radix Attention

In step (5), the second chat session continues. However, due to the memory limit, node "c" from (4) must be evicted

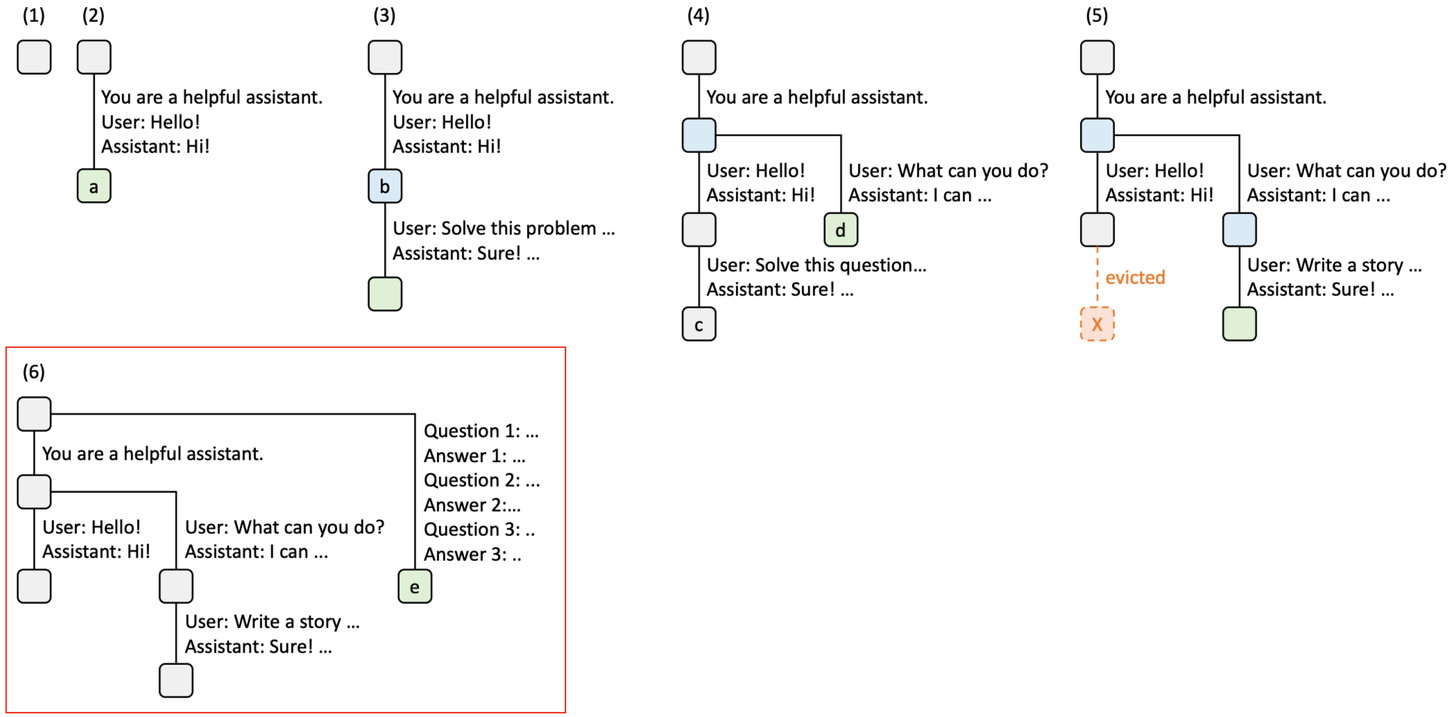
The new turn is appended after node "d" in (4)



Radix Attention

In step (6), the server receives a few-shot learning query, processes it, and inserts it into the tree

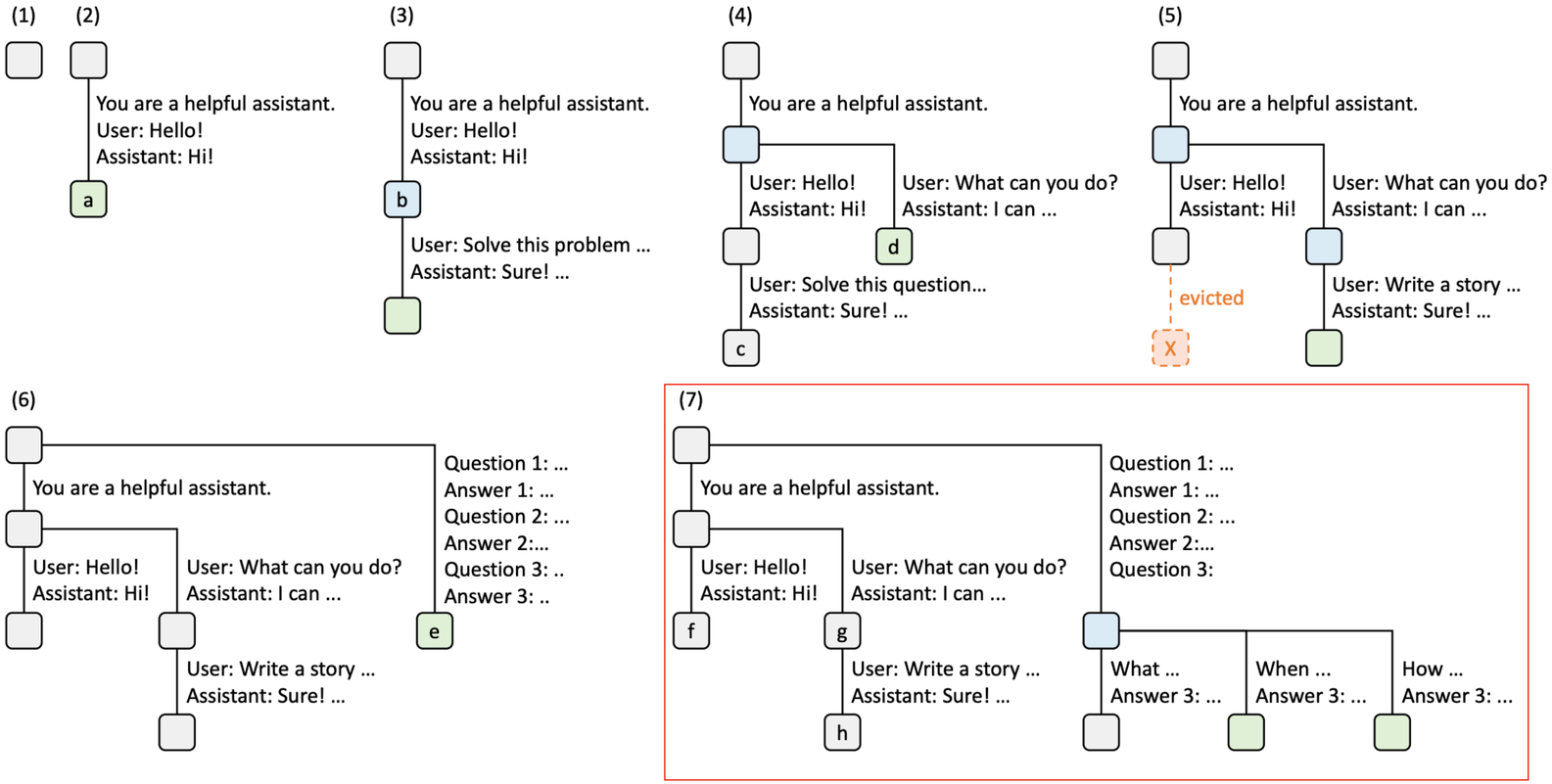
The root node is split because the new query does not share any prefix with existing nodes



Radix Attention

In step (7), the server receives a batch of additional few-shot learning queries

These queries share the same set of few-shot examples, so we split node ’e’ from (6) to enable sharing

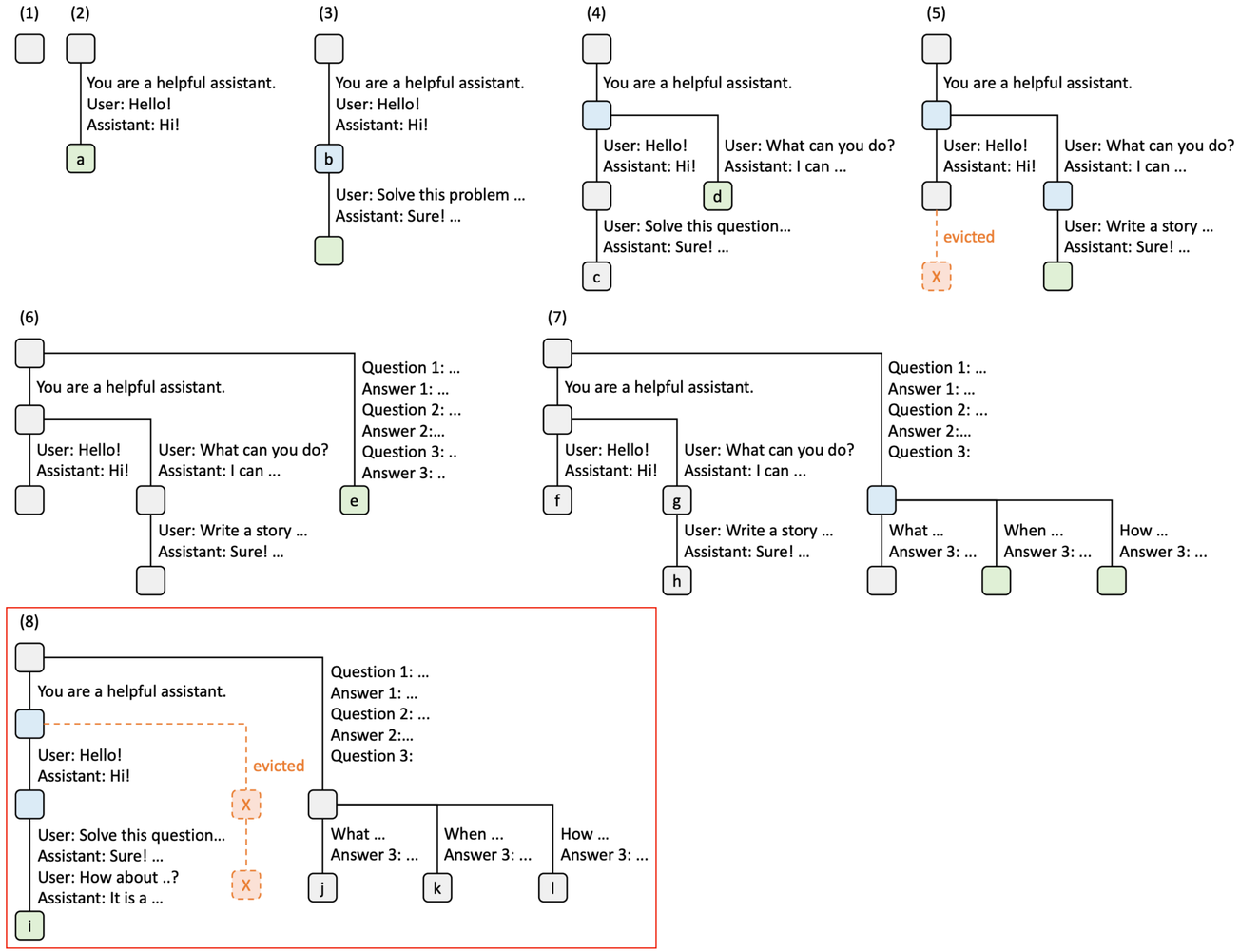


https://arxiv.org/abs/2312.07104

Radix Attention

In step (8), the server receives a new message from the first chat session

It evicts all nodes from the second chat session (node "g" and "h") as they are least recently used

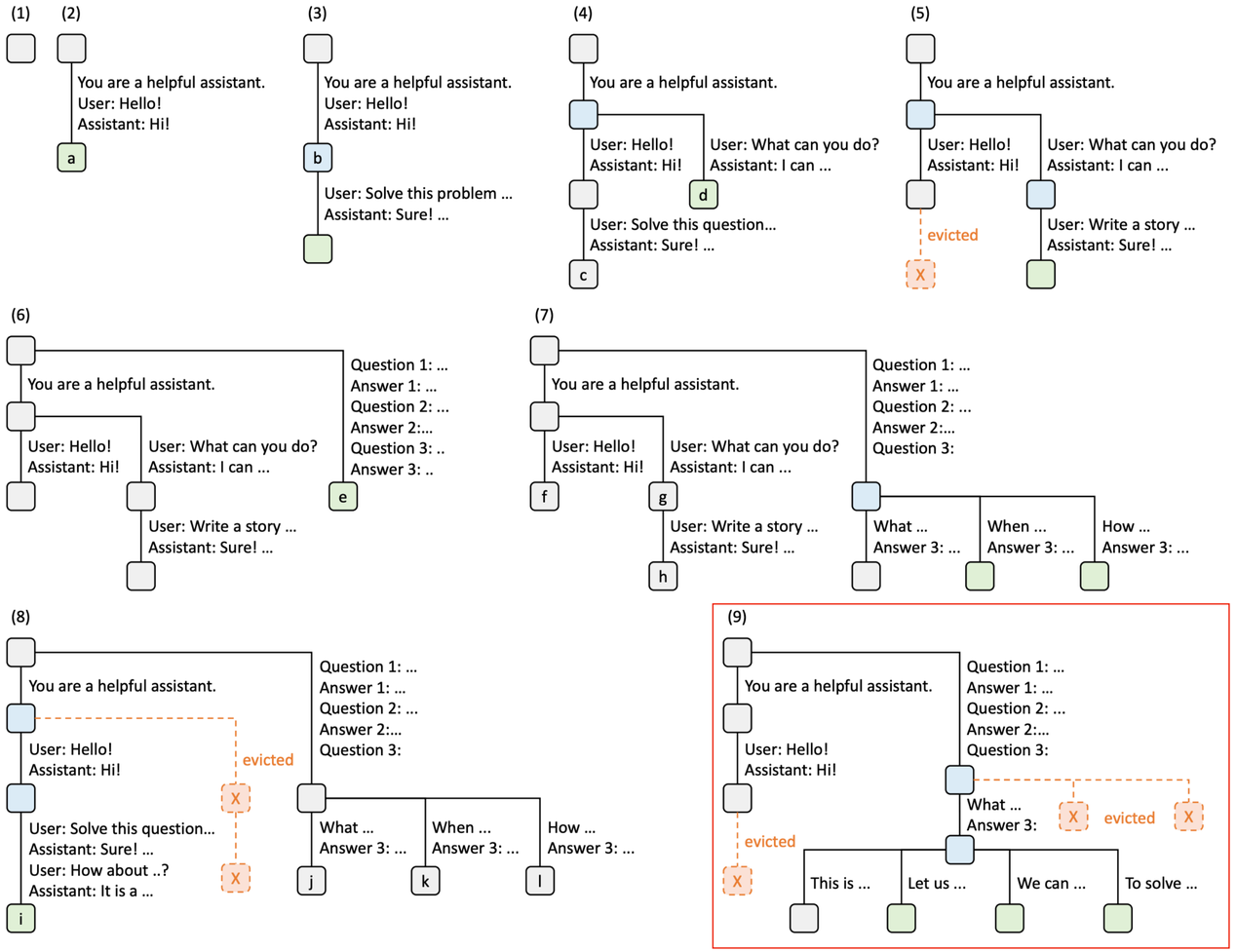


https://arxiv.org/abs/2312.07104

Radix Attention

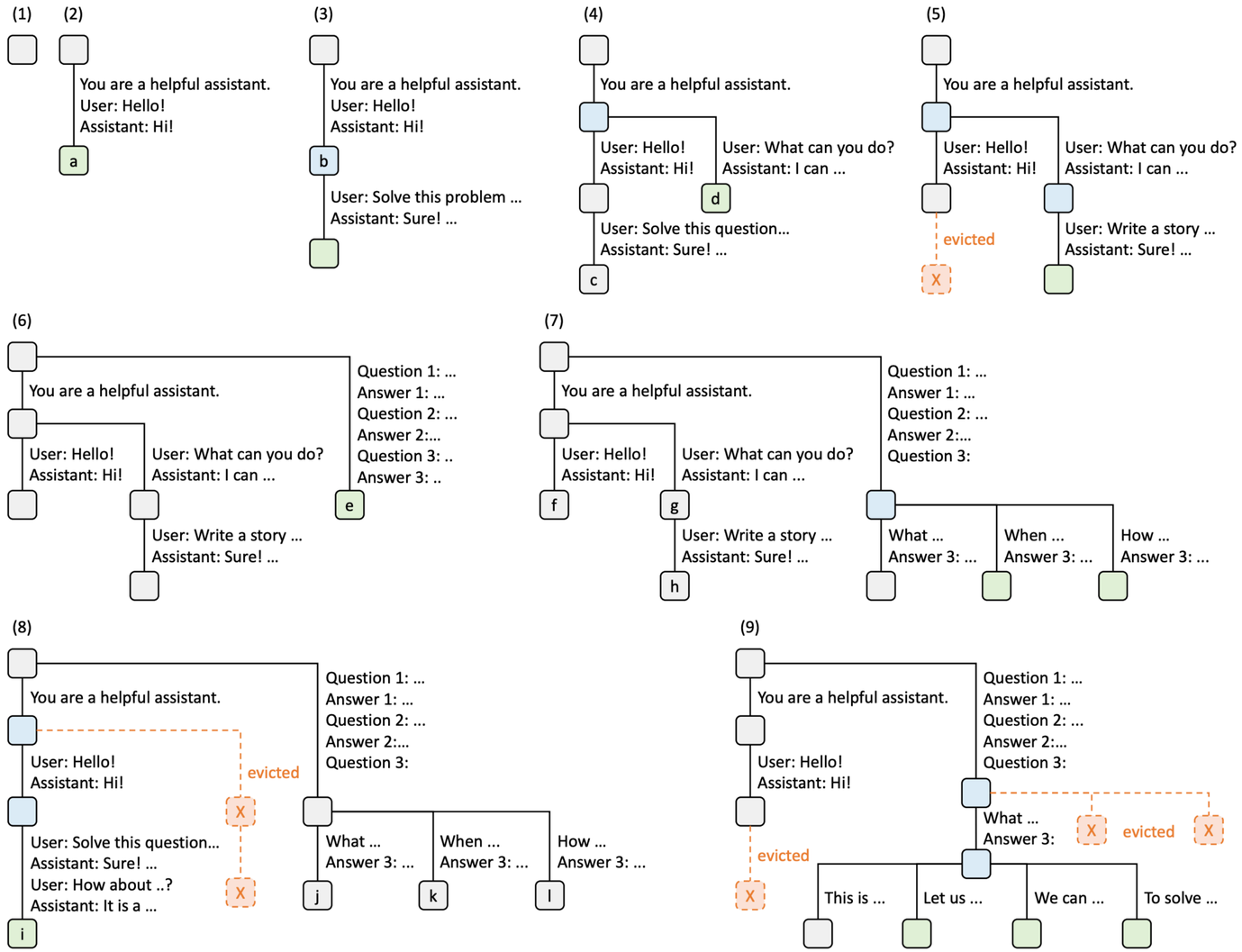
In step (9), the server receives a request to sample more answers for the questions in node "j" from (8), likely for self-consistency prompting

Evict node "i", "k", and "l" in (8)



https://arxiv.org/abs/2312.07104

Radix Attention



https://arxiv.org/abs/2312.07104

Model serving

Distillation

A larger teacher supervises a smaller student

The teacher provides richer training signals than a single correct label

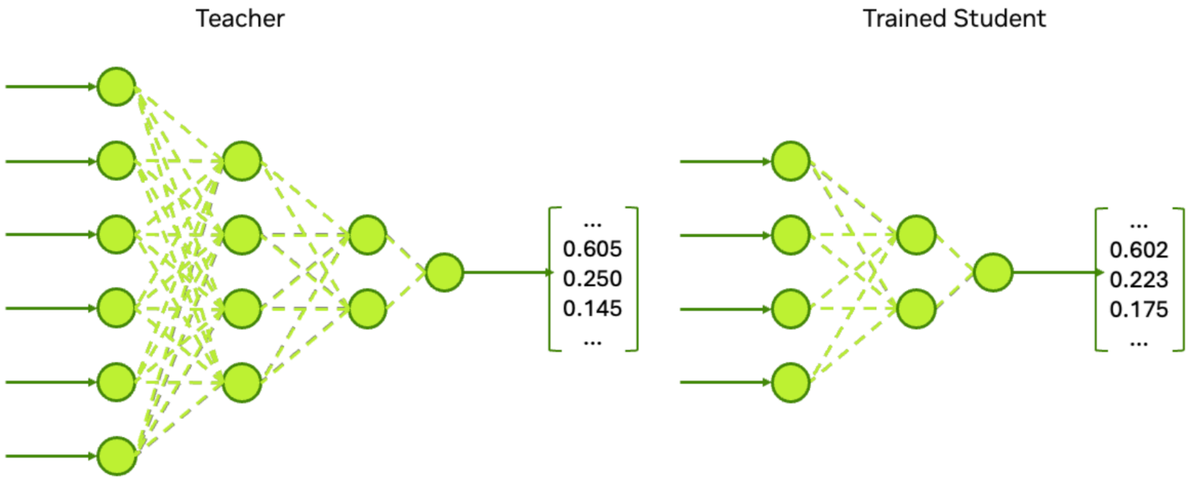
- **Output distributions:** learn which alternative tokens the teacher considers plausible
- **Generated responses:** train on answers or reasoning traces produced by the teacher
- **Hidden representations:** match selected intermediate features, using projections when dimensions differ

Example: logit distillation

$$p_T^\tau = \mathrm{softmax}(z_T/\tau), \qquad p_S^\tau = \mathrm{softmax}(z_S/\tau) \implies \mathscr{L} = (1-\alpha)\mathscr{L}_{\mathrm{CE}} + \alpha\tau^2 D_{\mathrm{KL}}\left(p_T^\tau \,\|\, p_S^\tau\right)$$

Temperature τ controls distribution softness. Weight α balances the training objectives

The teacher is used during training. Only the student is needed at inference



Continuous batching

Requests arrive at different times and generate different numbers of tokens

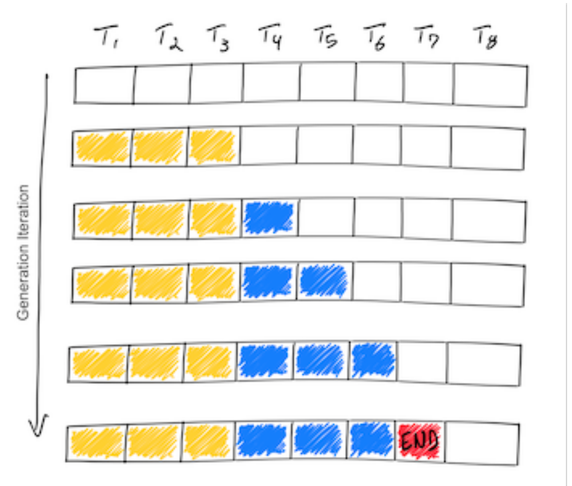
Batching lets multiple requests share the cost of reading model weights

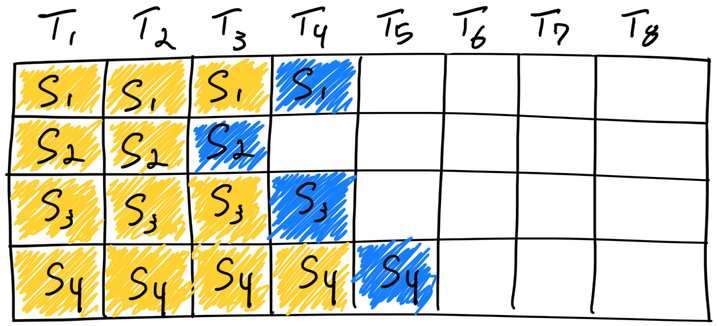
Static batching — the batch keeps the same requests until completion. Finished requests leave unused capacity

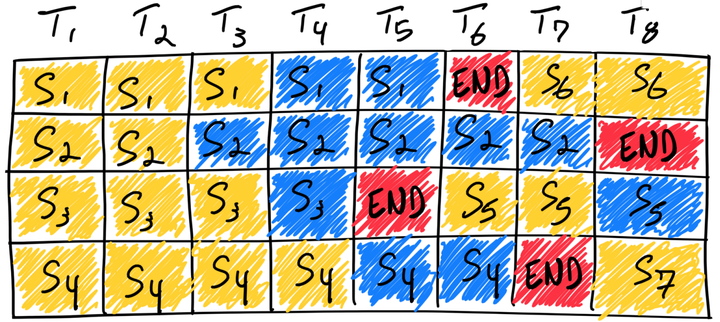
Continuous batching —the scheduler updates the active batch between model iterations:

- Remove completed requests
- Admit waiting requests when resources allow
- Continue generation for active requests

Higher utilization can improve throughput and reduce queueing under load







https://www.anyscale.com/blog/continuous-batching-llm-inference

Prefix caching

Requests with an identical prefix can reuse its computed KV states

Compute new states for the suffix while attending to the cached prefix

- Saves repeated prefill computation
- Can reduce time to first token
- Cached blocks consume memory and may be evicted

Reuse requires matching token prefixes and compatible model state

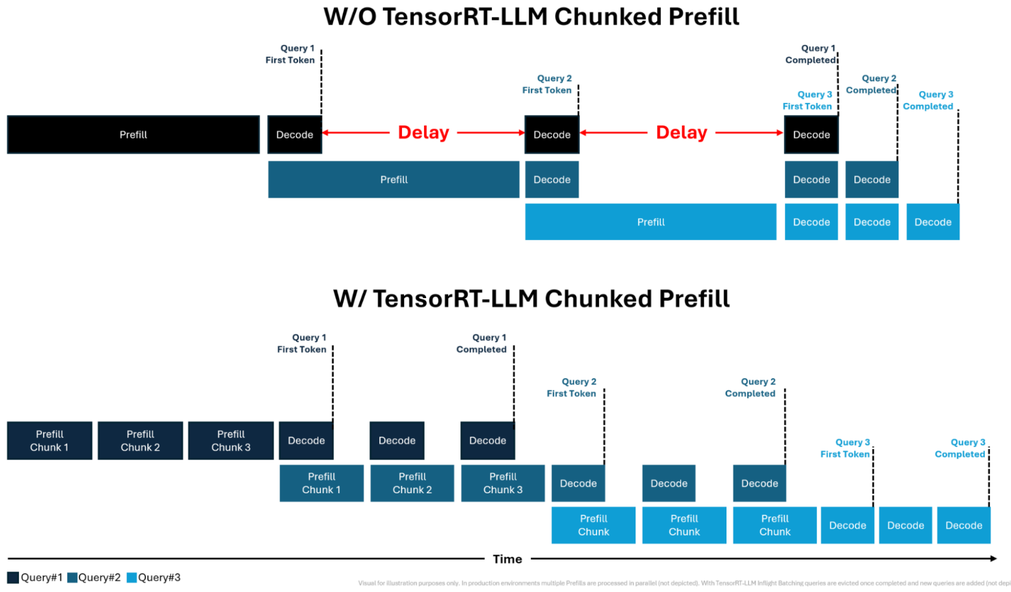
The prefix remains part of the context during decoding

Example: 8,000 cached prefix tokens + 200 new tokens

Request	Shared prefix	New suffix
A	System prompt + document	“Summarize the document”
B		“List the main risks”
C		“Find the payment terms”

Chunked prefill

- A long prefill can delay the next tokens of active requests
- Chunked prefill divides prompt processing across multiple scheduler iterations
- Later chunks attend to the KV states produced by earlier chunks
  - **Smaller chunks:** shorter interruptions to ongoing generation
  - **Larger chunks:** more prefill progress per iteration, often better GPU efficiency
- Chunk size balances TTFT, inter-token latency, and throughput



https://developer.nvidia.com/blog/streamlining-ai-inference-performance-and-deployment-with-nvidia-tensorrt-llm-chunked-prefill/

Disaggregated inference

Prefill and decode run on separate GPU worker pools

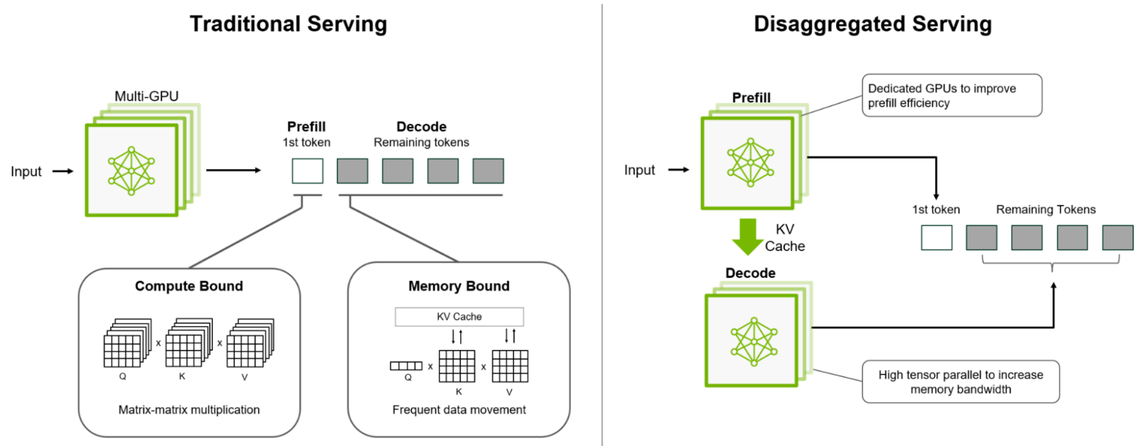
1. **Prefill workers** process the prompt and produce KV states
2. **KV transfer** moves these states to a decode worker
3. **Decode workers** continue autoregressive generation

Why separate them?

- Prevent long prefills from interrupting ongoing decoding
- Scale prefill and decode capacity independently
- Tune batching and model parallelism for each phase

The trade-off: KV transfer, additional model replicas, and load balancing

Better latency isolation can improve throughput within latency targets, but the benefit depends on workload and interconnect performance



Dealing with quadratic attention

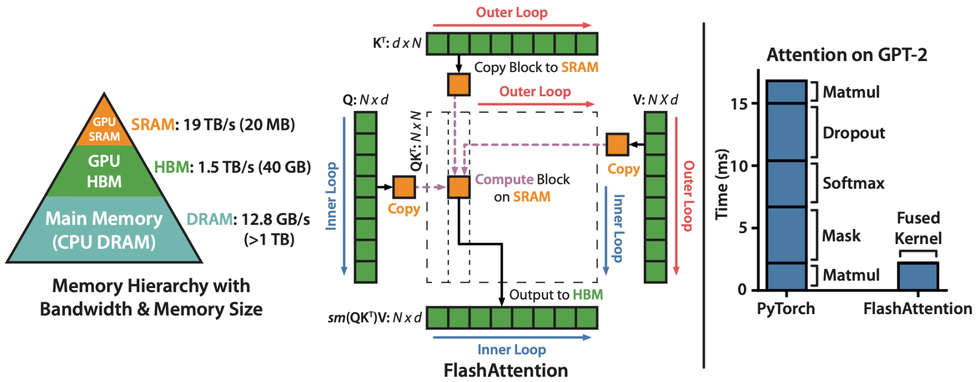
Flash attention

Standard attention computes:

$$S = \frac{QK^\top}{\sqrt{d}}, \quad P = \mathrm{softmax}(S), \quad O = PV$$

A straightforward implementation writes intermediate matrices to GPU memory:

- Write attention scores S
- Read S, compute softmax, and write probabilities P
- Read P to compute the output



https://arxiv.org/abs/2205.14135

Flash attention

FlashAttention processes small blocks that fit in fast on-chip memory

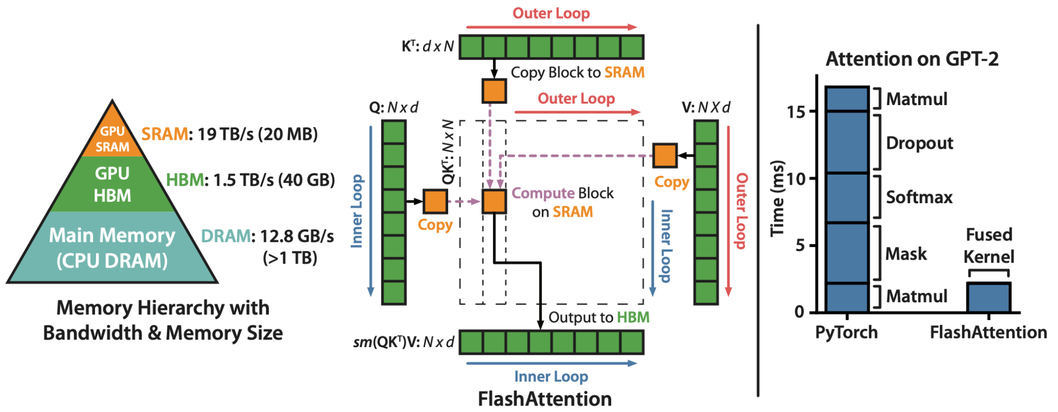
1. Load a query tile and a key/value tile
2. Compute scores and apply the attention mask
3. Update running softmax statistics and the output accumulator
4. Continue with the next key/value tile

For each query row, maintain:

$$m = \max_j s_j, \quad \ell = \sum_j e^{s_j - m}, \quad u = \sum_j e^{s_j - m} v_j \implies o = \frac{u}{\ell}$$

When the running maximum changes, rescale earlier contributions

The full score and probability matrices never need to be stored in HBM



https://arxiv.org/abs/2205.14135